

Capa, contracapa.



**AUTARQUIA DE ENSINO SUPERIOR DE ARCOVERDE
CENTRO DE ENSINO SUPERIOR DE ARCOVERDE
CURSO SUPERIOR EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Assetra - Plataforma de Gestão de Ativos de Tecnologia

Relatório Técnico

KELVIN KAUÊ VIANA SIQUEIRA

KELVIN KAUÊ VIANA SIQUEIRA

Assetra - Plataforma de Gestão de Ativos de Tecnologia

Relatório Técnico apresentado como requisito parcial para a disciplina de Projeto de Desenvolvimento de Sistemas Web / Projeto de Desenvolvimento de Sistema Corporativo, do curso superior em Análise em Desenvolvimento de Sistemas, sob orientação do Profº Dennys Cavalcanti Carvalho.

HISTÓRICO DE REVISÃO

Data	Versão	Descrição
10/03/2026	1.0	Elaboração dos primeiros conteúdos para implementação no documento.
19/03/2026	1.1	Implementação dos requisitos funcionais, não funcionais e regras de negócios.
24/03/2026	1.2	Criação dos diagramas de caso de uso, classe e sequência.
16/04/2026	1.3	Ajuste de ERS e ajuste no diagrama de classe.
30/04/2026	1.4	Modelagem do sistema finalizado. Adição dos wireframes e mockups.
11/05/2026	1.5	Fluxo de navegação implementado e início da documentação técnica.
12/05/2026	1.6	Finalização da documentação técnica e implementação do manual do usuário.
14/05/2026	1.7	Referências Bibliográficas e Apêndice completos.

SUMÁRIO

1 DOCUMENTO DE DEFINIÇÃO DE ESCOPO (DDE)	5
1.1. INTRODUÇÃO	5
1.2. VISÃO GERAL DO DOCUMENTO	5
1.3. IDENTIFICAÇÃO DO PROJETO	5
1.4. OBJETIVOS DO PROJETO	5
1.5. JUSTIFICATIVA	6
1.6. IDENTIFICAÇÃO DOS REQUISITOS	7
1.6.1. Prioridades dos Requisitos	7
1.7. ESCOPO DO PRODUTO E ENTREGÁVEIS	7
1.7.1. Funcionalidades Previstas	7
1.7.2. Entregáveis	8
1.8. PREMISSAS E RESTRIÇÕES	8
1.8.1. Premissas	8
1.8.2. Restrições	8
1.9. CRITÉRIOS DE ACEITAÇÃO DO PROJETO	8
1.10. EXCLUSÕES DO ESCOPO	9
1.11. STAKEHOLDERS ENVOLVIDOS	9
1.12. RISCOS INICIAIS	9
2 DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS (ERS)	10
2.1. REQUISITOS FUNCIONAIS	10
2.2. REQUISITOS NÃO FUNCIONAIS	13
2.3. REGRAS DE NEGÓCIO	15
3 DOCUMENTO DE ESPECIFICAÇÃO DE MODELAGEM (DEM)	16
3.1 MODELAGEM DE DADOS	16
3.1.1 Entidade-Relacionamento	16
3.1.2 Dicionário de Dados	16
3.2 MODELAGEM COMPORTAMENTAL	19
3.2.1 Diagrama de Sequência	19
3.2.2 Diagrama de Estados	19
3.3. MODELAGEM ESTRUTURAL	20
3.3.1 Diagrama de Caso de Uso	20
3.3.2 Diagrama de Classes	21
3.3.3 Diagrama de Componentes	24
3.3.4 Diagrama de Arquitetura	25
3.4 MAPEAMENTO OBJETO-RELACIONAL (ORM)	26
3.5 BPMN (BUSINESS PROCESS MODEL AND NOTATION)	27
4 DOCUMENTO DE ESPECIFICAÇÃO DE INTERFACES (DEI)	31
4.1. WIREFRAMES	31
4.2. MOCKUPS	32
4.3. FLUXO DE NAVEGAÇÃO	41
5 DOCUMENTAÇÃO TÉCNICA	44
5.1. ARQUITETURA DO SISTEMA	44

5.1.1. Segmentação da Arquitetura.....	45
5.2. TECNOLOGIAS UTILIZADAS.....	46
5.2.1 Frontend.....	46
5.2.2. Backend.....	46
5.2.3. Banco de Dados.....	47
5.2.4. Ferramentas de Apoio.....	47
5.2.5. Padrões Adotados.....	47
5.2.6. Boas Práticas e Convenções.....	48
5.2.7. Requisitos de Infraestrutura.....	49
5.2.8. APIs e Integrações.....	50
5.2.9. Caracterização da API.....	50
5.3. REPOSITÓRIO E CÓDIGO-FONTE.....	50
5.3.1. Localização.....	50
5.3.2. Estrutura de Pastas.....	51
5.3.3. Conteúdo das Pastas Principais.....	51
6. MANUAL DO USUÁRIO.....	53
7. REFERÊNCIAS.....	53
8. APÊNDICE.....	53

DDE

1 DOCUMENTO DE DEFINIÇÃO DE ESCOPO (DDE)

1.1 INTRODUÇÃO

O presente projeto descreve o desenvolvimento do Assetra, uma plataforma web corporativa voltada para a gestão de ativos de tecnologia da informação dentro de organizações. O sistema tem como objetivo permitir que empresas registrem, monitorem e gerenciem equipamentos tecnológicos como computadores, servidores, dispositivos de rede e outros recursos de infraestrutura de TI.

A plataforma permitirá o controle do ciclo de vida dos ativos, incluindo cadastro, movimentação, manutenção e registro de ocorrências. Dessa forma, busca-se melhorar a organização dos recursos tecnológicos das empresas, oferecendo maior visibilidade sobre o uso e estado dos equipamentos.

1.2 VISÃO GERAL DO DOCUMENTO

Este documento apresenta o escopo detalhado do Assetra, uma plataforma corporativa de gestão de ativos de TI.

O objetivo deste documento é definir as diretrizes que orientarão o desenvolvimento do sistema, incluindo objetivos, funcionalidades previstas, restrições técnicas e critérios de aceitação. Além disso, o documento serve como referência para o planejamento e implementação do projeto ao longo da disciplina.

1.3 IDENTIFICAÇÃO DO PROJETO

Nome do Projeto: Assetra – Plataforma de Gestão de Ativos de TI

Autor: Kelvin Kauê Viana Siqueira

1.4 OBJETIVOS DO PROJETO

Objetivo Geral: Desenvolver uma plataforma web corporativa para gestão de ativos de tecnologia da informação, permitindo que organizações registrem, monitorem e gerenciem seus equipamentos de TI de forma centralizada.

Objetivos Específicos:

- Levantar e documentar os requisitos funcionais e não funcionais do sistema.
- Projetar a arquitetura do sistema utilizando boas práticas de engenharia de software.
- Desenvolver um sistema web para cadastro e gerenciamento de ativos de TI.
- Implementar mecanismos de controle de usuários e permissões de acesso.
- Desenvolver dashboards com indicadores sobre o estado e utilização dos ativos.
- Implementar APIs para integração com outros sistemas corporativos.
- Realizar testes de funcionalidade e integração do sistema.

1.5 JUSTIFICATIVA

A gestão de ativos de TI é um processo essencial para organizações que dependem de recursos tecnológicos em suas operações. A ausência de controle adequado pode gerar perdas financeiras, falhas operacionais e dificuldades na tomada de decisão. Nesse contexto, os sistemas de informação desempenham papel fundamental, sendo definidos como sistemas que “coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisões e o controle em uma organização” (LAUDON, 2004, p. 4).

Além disso, a utilização de sistemas de informação contribui diretamente para a melhoria da eficiência operacional e da gestão organizacional. Segundo Laudon e Laudon (2014, p. 12), “os sistemas de informação são essenciais para alcançar a excelência operacional, desenvolver novos produtos e serviços e melhorar a tomada de decisões”.

Outro ponto relevante é que os recursos de tecnologia da informação são considerados estratégicos para o funcionamento das organizações, sendo necessário o uso de processos estruturados para seu controle e gerenciamento. De acordo com Andrade et al. (2016, p. 2), “a gestão de mudanças em ambientes de TI contribui para reduzir falhas operacionais e melhorar a eficiência dos processos organizacionais”.

Nesse contexto, o desenvolvimento do sistema Assetra se justifica pela necessidade de centralizar, organizar e monitorar ativos de TI em um ambiente corporativo. A solução proposta busca melhorar a eficiência operacional, reduzir erros manuais e fornecer suporte à tomada de decisão por meio de informações estruturadas e atualizadas.

1.6 IDENTIFICAÇÃO DOS REQUISITOS

Por convenção, os requisitos são referenciados pelo nome da subseção onde estão descritos, seguido do seu identificador, conforme o esquema abaixo:

- O requisito funcional [Cadastro de Usuários.RF-01] está localizado na subseção “Requisitos Funcionais”, dentro do bloco identificado como [RF-01].
- O requisito não funcional [Disponibilidade.NF-04] encontra-se na seção “Requisitos Não Funcionais de Confiabilidade”, no bloco identificado como [NF-04].

1.6.1 Prioridades dos Requisitos

Os requisitos do sistema são classificados em três níveis de prioridade:

- Essencial: indispensável para o funcionamento do sistema. Sem ele, o sistema não opera. Deve ser obrigatoriamente implementado.
- Importante: afeta a qualidade do funcionamento. O sistema pode ser utilizado sem esse requisito, mas de forma insatisfatória. Sua implementação é recomendada.
- Desejável: não interfere nas funcionalidades básicas. O sistema funciona bem sem ele. Pode ser incluído em versões futuras, caso não haja tempo para implementá-lo na versão atual.

1.7 ESCOPO DO PRODUTO E ENTREGÁVEIS

1.7.1 Funcionalidades Previstas

O sistema Assetra deverá oferecer as seguintes funcionalidades principais:

- Cadastro e gerenciamento de usuários.
- Cadastro e gerenciamento de empresas.
- Registro e gerenciamento de ativos de tecnologia.
- Registro de movimentações de ativos entre usuários ou setores.
- Registro de manutenções realizadas em equipamentos.
- Registro de ocorrências relacionadas aos ativos.
- Visualização de dashboards com indicadores sobre os ativos cadastrados.

- Controle de permissões de acesso baseado em papéis de usuários.
- Registro de histórico e auditoria de alterações no sistema.

1.7.2 Entregáveis

Os seguintes itens serão entregues ao final do projeto:

- Código fonte completo do sistema em repositório Git.
- Relatório técnico do projeto incluindo requisitos, documentação técnica, as modelagem e manual do usuário.
- Sistema funcional implantado em ambiente de teste.

1.8 PREMISSAS E RESTRIÇÕES

1.8.1 Premissas

- Os usuários terão acesso à internet para utilizar a plataforma.
- As organizações possuirão equipamentos de TI que necessitam ser gerenciados.
- Os usuários terão conhecimentos básicos de informática para operar o sistema

1.8.2 Restrições

- O sistema será desenvolvido apenas como aplicação web.
- O projeto deverá ser concluído dentro do prazo definido pela disciplina.
- As tecnologias utilizadas incluirão HTML, CSS, JS, Vue.js de frameworks web e banco de dados relacional e não relacional(NoSQL).

1.9 CRITÉRIOS DE ACEITAÇÃO DO PROJETO

Para que o projeto seja considerado aprovado, o sistema deverá atender aos seguintes critérios:

- Implementação das funcionalidades essenciais definidas no escopo.
- Interface web funcional e responsiva.
- Sistema capaz de realizar operações de cadastro, edição e consulta de dados.
- Implementação de autenticação e controle de acesso de usuários.
- Disponibilização de dashboards com indicadores sobre os ativos cadastrados.
- Código organizado e documentado de acordo com boas práticas de desenvolvimento.

1.10 EXCLUSÕES DO ESCOPO

Os seguintes itens não farão parte da versão inicial do sistema:

- Desenvolvimento de aplicativo mobile nativo.
- Integração direta com sistemas externos corporativos.
- Controle financeiro detalhado relacionado aos ativos.

1.11 STAKEHOLDERS ENVOLVIDOS

Organizadores: Aluno responsável pelo desenvolvimento do sistema.

Participantes: Usuários das empresas que utilizarão o sistema para gerenciar ativos.

Administrador do sistema: Responsável pela administração da plataforma e gerenciamento de usuários.

Professor orientador: Professor responsável pelo acompanhamento e avaliação do projeto.

Testers: Usuários convidados para validar o funcionamento do sistema durante os testes.

1.12 RISCOS INICIAIS

Alguns fatores podem impactar o andamento do projeto:

- Prazo limitado para desenvolvimento do sistema.
- Possíveis dificuldades técnicas com tecnologias utilizadas.
- Necessidade de aprendizado de novas ferramentas durante o desenvolvimento.
- Eventuais problemas de integração entre frontend e backend.

ERS

2 DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS (ERS)

2.1 REQUISITOS FUNCIONAIS

Requisitos funcionais são as funções que usuários e clientes esperam que o software ofereça. Eles estão diretamente ligados às funcionalidades que o sistema deve fornecer. Segundo Ian Sommerville, requisitos funcionais definem as funcionalidades específicas que um sistema de software deve realizar para atender às necessidades dos usuários e alcançar os objetivos do projeto.

CONTEXTO: USUÁRIO E ACESSO

- [RF-01] Cadastro de Usuários:
O sistema deve permitir o cadastro de novos usuários na plataforma.

Atores: Administrador.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

➤ Critério de aceitação: O administrador preenche os campos obrigatórios (nome, e-mail, senha e papel do usuário) e o sistema confirma a criação do usuário.

- [RF-02] Autenticação de Usuários:
O sistema deve permitir que usuários cadastrados realizem login na plataforma.

Atores: Usuário.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

➤ Critério de aceitação: O usuário informa e-mail e senha válidos e o sistema permite acesso ao dashboard.

- [RF-03] Controle de Permissões:
O sistema deve controlar o acesso às funcionalidades com base no papel do usuário.

Atores: Administrador.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

➤ Critério de aceitação: Usuários visualizam apenas as funcionalidades permitidas para seu nível de acesso.

CONTEXTO: ORGANIZAÇÃO

- [RF-04] Cadastro de Empresas:

O sistema deve permitir registrar empresas que utilizarão a plataforma.

Atores: Administrador.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O administrador informa os dados da empresa e o sistema registra a nova organização na plataforma.

CONTEXTO: ATIVOS

- [RF-05] Cadastro de Ativos de TI:

O sistema deve permitir registrar ativos tecnológicos como computadores, servidores, impressoras e dispositivos de rede.

Atores: Administrador, Técnico.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O usuário preenche os dados do ativo (nome, tipo, número de série, localização e responsável) e o sistema confirma o cadastro.

- [RF-06] Consulta de Ativos:

O sistema deve permitir visualizar e pesquisar os ativos cadastrados.

Atores: Administrador, Técnico, Gestor.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O sistema exibe uma lista de ativos com informações básicas e permite filtrar por tipo, status ou responsável.

- [RF-07] Atualização de Ativos:

O sistema deve permitir atualizar as informações de ativos cadastrados.

Atores: Administrador, Técnico.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: O usuário altera os dados de um ativo e o sistema registra a atualização.

- [RF-08] Dashboard de Ativos:

O sistema deve apresentar um painel com indicadores sobre os ativos cadastrados.

Atores: Administrador, Gestor.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de aceitação: O sistema apresenta gráficos ou indicadores com informações como quantidade de ativos, ativos por tipo e ativos em manutenção.

CONTEXTO OPERAÇÕES DE ATIVOS

- [RF-09] Registro de Movimentação de Ativos:
O sistema deve permitir registrar movimentações de ativos entre usuários ou setores.

Atores: Administrador, Técnico.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

- Critério de aceitação: Ao alterar responsável ou localização de um ativo, o sistema registra automaticamente o histórico da movimentação.

- [RF-10] Registro de Manutenção:
O sistema deve permitir registrar manutenções realizadas em ativos.

Atores: Técnico.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de aceitação: O usuário registra informações da manutenção (tipo, descrição, data e responsável) e o sistema salva o registro.

- [RF-11] Registro de Ocorrências:
O sistema deve permitir registrar incidentes ou problemas relacionados aos ativos.

Atores: Usuário, Técnico.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de aceitação: O usuário registra uma ocorrência informando o ativo e a descrição do problema.

CONTEXTO: AUDITORIA

- [RF-12] Registro de Auditoria:
O sistema deve registrar alterações importantes realizadas na plataforma.

Atores: Sistema.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

- Critério de aceitação: Toda alteração relevante gera um registro de histórico contendo usuário, data e ação realizada.

2.2 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais representam atributos de qualidade que um software deve possuir, mesmo que não estejam diretamente ligados às funcionalidades principais. Eles indicam características esperadas em qualquer sistema profissionalmente desenvolvido, como desempenho, segurança, usabilidade e confiabilidade. Além disso, expressam restrições técnicas que o software precisa atender, garantindo que ele funcione de forma eficiente dentro de seu ambiente operacional.

CONTEXTO: SEGURANÇA E ACESSO

- [NF-01] Segurança de Autenticação:
O sistema deve garantir que apenas usuários autenticados possam acessar as funcionalidades da plataforma.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

➤ Critério de medição: Usuários devem informar credenciais válidas (e-mail e senha) para acessar o sistema. Caso contrário, o acesso é negado..

- [NF-02] Controle de Acesso por Papéis:
O sistema deve restringir funcionalidades de acordo com o papel do usuário.

Prioridade: ☒ Essencial ☐ Importante ☐ Desejável

➤ Critério de medição: Usuários visualizam apenas funcionalidades permitidas para seu nível de acesso (ex.: administrador, gestor ou técnico).

CONTEXTO: DESEMPENHO E DISPONIBILIDADE

- [NF-03] Desempenho do Sistema:
O sistema deve responder às requisições do usuário em tempo adequado durante a utilização.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

➤ Critério de medição: As páginas principais do sistema devem carregar em tempo inferior a 3 segundos em condições normais de uso.

- [NF-04] Disponibilidade do Sistema:
O sistema deve estar disponível para acesso através da internet.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

➤ Critério de medição: Usuários conseguem acessar a plataforma por meio de um navegador web em ambiente online.

CONTEXTO: USABILIDADE E COMPATIBILIDADE

- [NF-05] Usabilidade da Interface:
A interface do sistema deve ser intuitiva e facilitar a navegação dos usuários.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

➤ Critério de medição: Usuários conseguem acessar as funcionalidades principais do sistema sem necessidade de treinamento avançado.
- [NF-06] Compatibilidade com Navegadores:
O sistema deve funcionar corretamente nos principais navegadores web modernos.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

➤ Critério de medição: O sistema deve funcionar corretamente em navegadores como Chrome, Edge e Firefox.

CONTEXTO: AUDITORIA

- [NF-07] Registro de Auditoria:
O sistema deve registrar logs das ações relevantes realizadas pelos usuários.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

➤ Critério de medição: O sistema registra informações como usuário, data e ação realizada em operações importantes.

CONTEXTO: INTEGRAÇÃO E DADOS

- [NF-08] Integrabilidade com Sistemas Externos:
O sistema deve disponibilizar APIs que permitam integração com outros sistemas corporativos.

Prioridade: ☐ Essencial ☐ Importante ☒ Desejável

➤ Critério de medição: Outros sistemas podem acessar informações do Assetra através de endpoints de API.
- [NF-09] Backup de Dados:
O sistema deve garantir a preservação das informações armazenadas por meio de mecanismos de backup.

Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

➤ Critério de medição: Os dados cadastrados no sistema devem ser armazenados de forma que possam ser recuperados em caso de falha ou perda de informações.

2.3 REGRAS DE NEGÓCIO

As regras de negócio definem os limites, comportamentos e restrições que regem o funcionamento da aplicação, garantindo conformidade com os objetivos organizacionais e legais. Abaixo estão listadas as principais regras aplicáveis ao sistema:

CONTEXTO: ATIVOS

- [RN-01] Associação de Ativos a Empresas:
Todo ativo cadastrado no sistema deve estar obrigatoriamente associado a uma empresa.
Prioridade: ☒ Essencial ☐ Importante ☐ Desejável
- [RN-02] Responsável pelo Ativo:
Um ativo pode possuir um usuário responsável associado a ele.
Prioridade: ☐ Essencial ☒ Importante ☐ Desejável
- [RN-03] Integridade dos Ativos:
Um ativo não pode ser removido do sistema caso possua registros de movimentação, manutenção ou ocorrências associados.
Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

CONTEXTO: OPERAÇÕES

- [RN-04] Registro Automático de Movimentações:
Sempre que um ativo tiver seu responsável ou localização alterados, o sistema deve registrar automaticamente uma movimentação.
Prioridade: ☒ Essencial ☐ Importante ☐ Desejável
- [RN-05] Histórico de Manutenção:
Um ativo pode possuir múltiplos registros de manutenção ao longo do seu ciclo de vida.
Prioridade: Prioridade: ☐ Essencial ☒ Importante ☐ Desejável
- [RN-06] Registro de Ocorrências:
As ocorrências registradas devem estar associadas a um ativo existente no sistema.
Prioridade: Prioridade: ☐ Essencial ☒ Importante ☐ Desejável

DEM

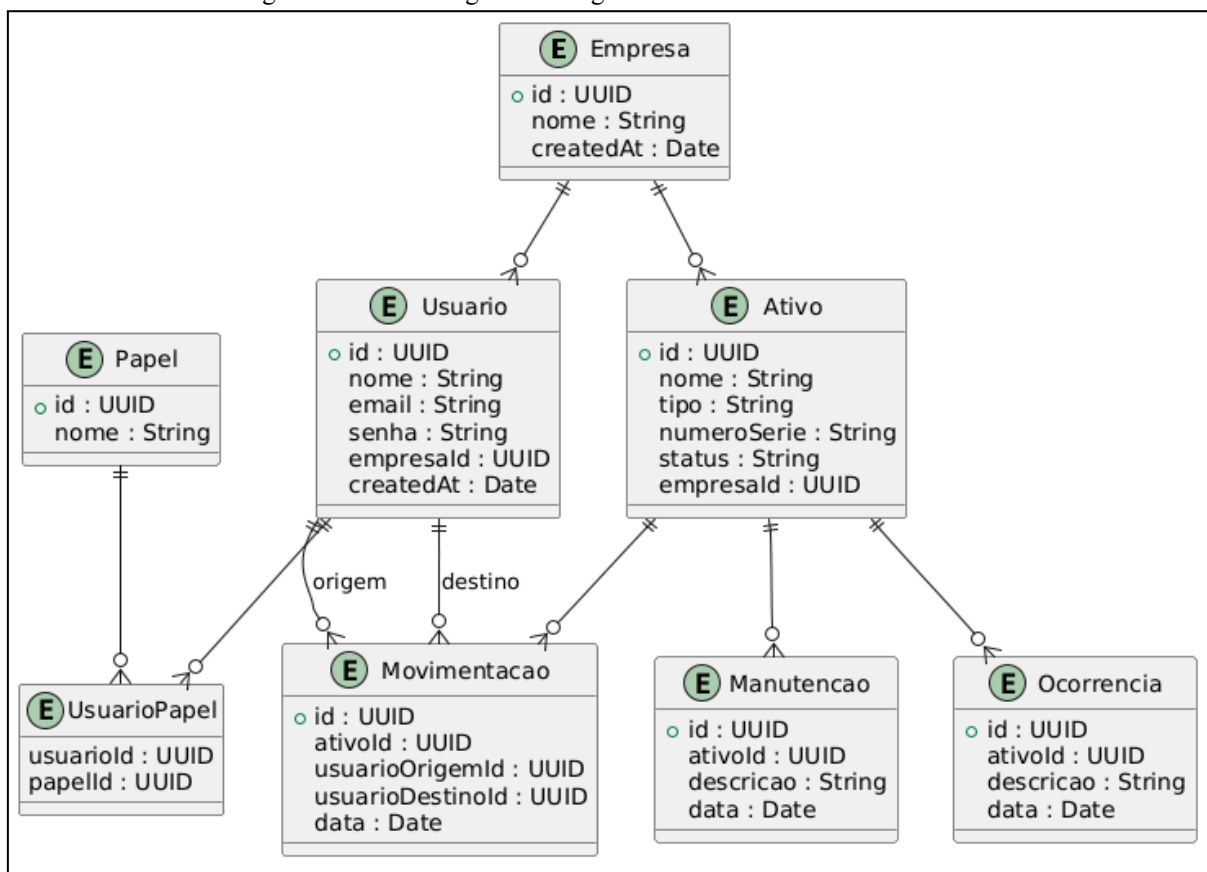
3 DOCUMENTO DE ESPECIFICAÇÃO DE MODELAGEM (DEM)

3.1 MODELAGEM DE DADOS

3.1.1 Entidade-Relacionamento

O diagrama entidade-relacionamento representa a estrutura do banco de dados do sistema Assetra, incluindo entidades como empresa, usuário, ativos e seus relacionamentos. Também são representadas entidades relacionadas ao controle operacional, como movimentações, manutenções e ocorrências, garantindo rastreabilidade das informações.

Figura 1 - Panorama geral do diagrama de Entidade-Relacionamento



Fonte: Produzido pelo autor.

3.1.2 Dicionário de Dados

O dicionário de dados descreve os atributos das entidades do sistema, apresentando seus tipos e significados, com o objetivo de padronizar e facilitar a compreensão da estrutura de dados utilizada.

CONTEXTO: USUÁRIO

Campo	Tipo	Descrição
id	UUID	Identificador único do usuário
nome	String	Nome completo do usuário
email	String	Email utilizado para login
senha	String	Senha criptografada
empresaId	UUID	Referência à empresa
createdAt	Date	Data de criação

Fonte: Produzido pelo autor.

CONTEXTO: EMPRESA

Campo	Tipo	Descrição
id	UUID	Identificador da empresa
nome	String	Nome da empresa
createdAt	Date	Data de cadastro

Fonte: Produzido pelo autor.

CONTEXTO: ATIVO

Campo	Tipo	Descrição
id	UUID	Identificador do ativo
nome	String	Nome do ativo
tipo	String	Tipo do ativo (ex: computador)
numeroSerie	String	Número de série
status	String	Estado atual do ativo
empresaId	UUID	Empresa proprietária

Fonte: Produzido pelo autor.

CONTEXTO: MOVIMENTAÇÃO

Campo	Tipo	Descrição
id	UUID	Identificador da movimentação
ativoId	UUID	Ativo movimentado
usuarioOrigemId	UUID	Usuário de origem
usuarioDestinoId	UUID	Usuário de destino
data	Date	Data da movimentação

Fonte: Produzido pelo autor.

CONTEXTO: MANUTENÇÃO

Campo	Tipo	Descrição
id	UUID	Identificador da manutenção
ativoId	UUID	Ativo relacionado
descricao	String	Descrição do serviço
data	Date	Data da manutenção

Fonte: Produzido pelo autor.

CONTEXTO: OCORRÊNCIA

Campo	Tipo	Descrição
id	UUID	Identificador da ocorrência
ativoId	UUID	Ativo relacionado
descricao	String	Descrição do problema
data	Date	Data do registro

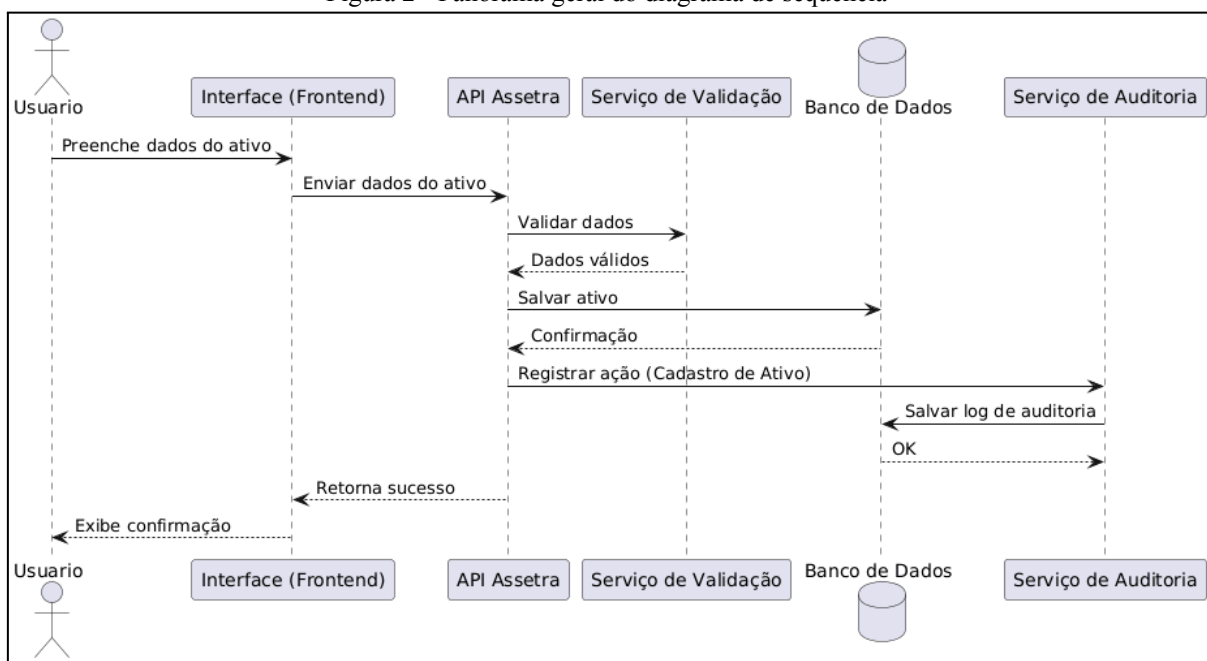
Fonte: Produzido pelo autor.

3.2 MODELAGEM COMPORTAMENTAL

3.2.1 Diagrama de Sequência

O diagrama de sequência representa o fluxo de execução do processo de cadastro de ativos no sistema Assetra. Nele são demonstradas as interações entre o usuário, a interface da aplicação, a API, os serviços internos e o banco de dados. O fluxo contempla etapas de validação, persistência dos dados e registro de auditoria, evidenciando o funcionamento interno da aplicação.

Figura 2 - Panorama geral do diagrama de sequência

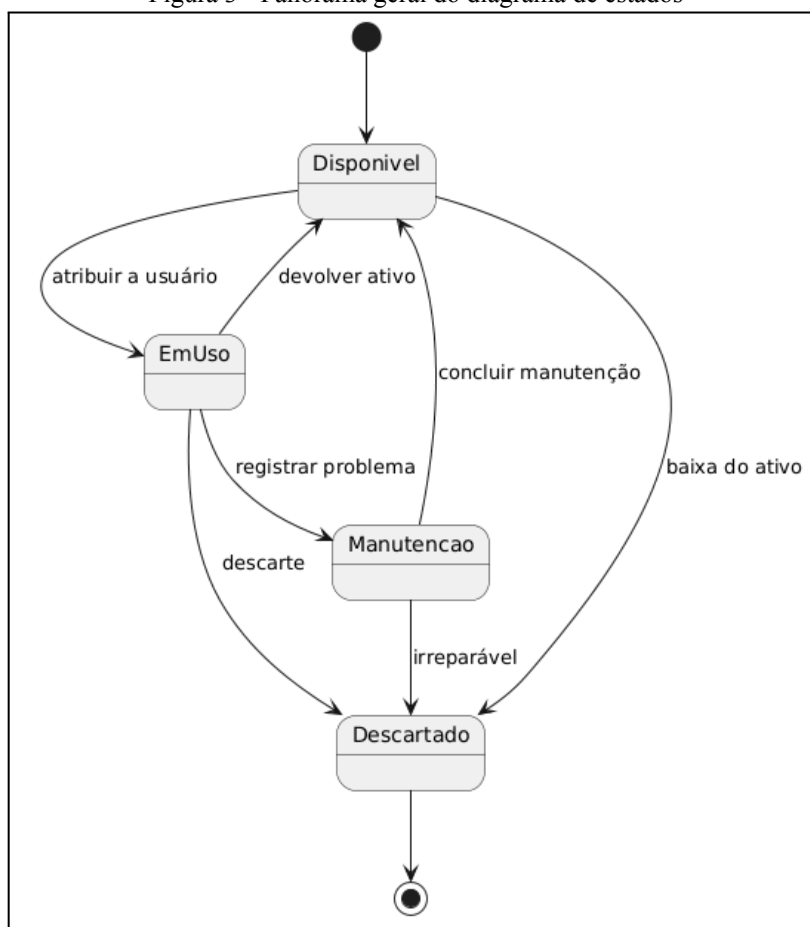


Fonte: Produzido pelo autor.

3.2.2 Diagrama de Estados

O diagrama de estados representa o ciclo de vida de um ativo dentro do sistema Assetra, evidenciando as possíveis transições entre estados como disponível, em uso, em manutenção e descartado.

Figura 3 - Panorama geral do diagrama de estados



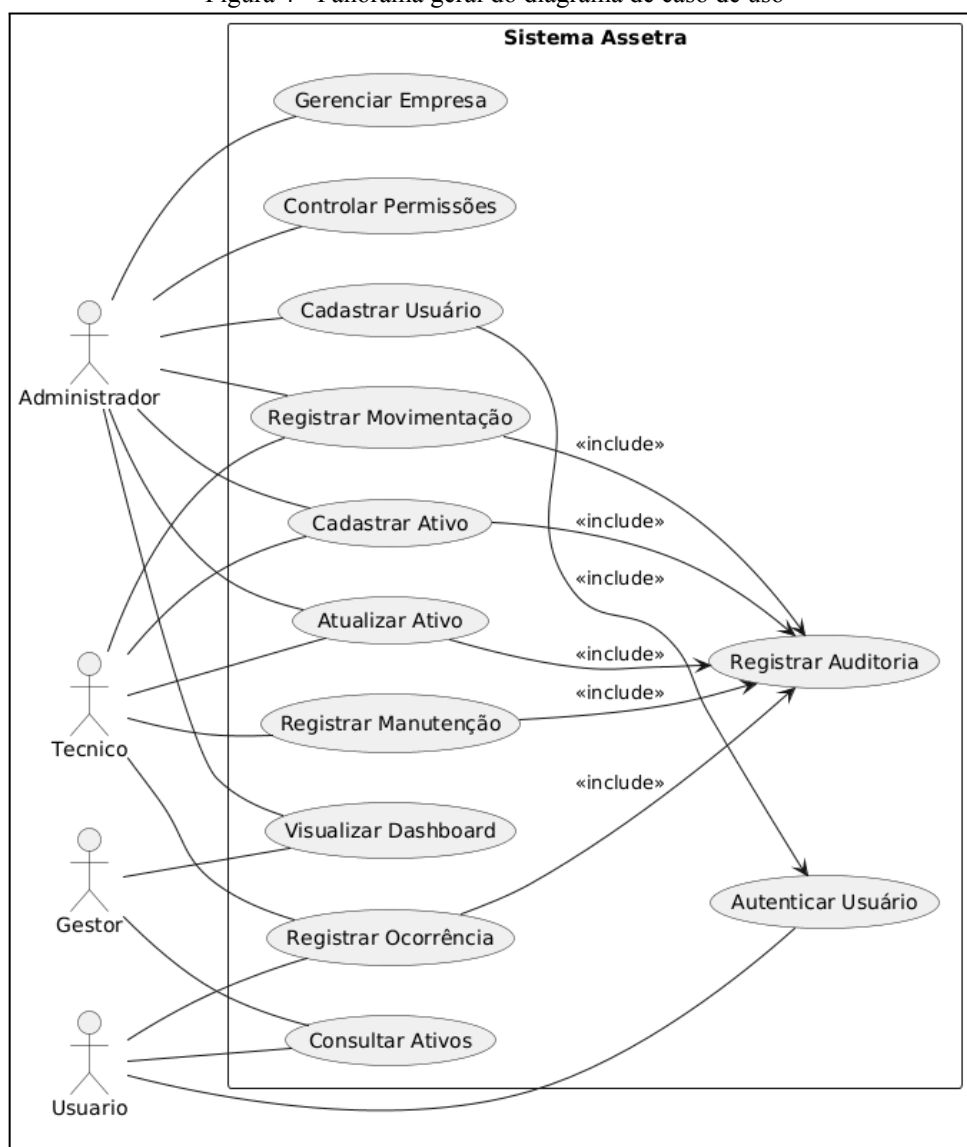
Fonte: Produzido pelo autor.

3.3 MODELAGEM ESTRUTURAL

3.3.1 Diagrama de Caso de Uso

O diagrama de caso de uso apresenta as principais funcionalidades do sistema Assetra, bem como as interações entre os diferentes atores e as operações disponíveis na plataforma. Também são representadas relações de inclusão entre casos de uso, evidenciando funcionalidades que são executadas de forma integrada.

Figura 4 - Panorama geral do diagrama de caso de uso

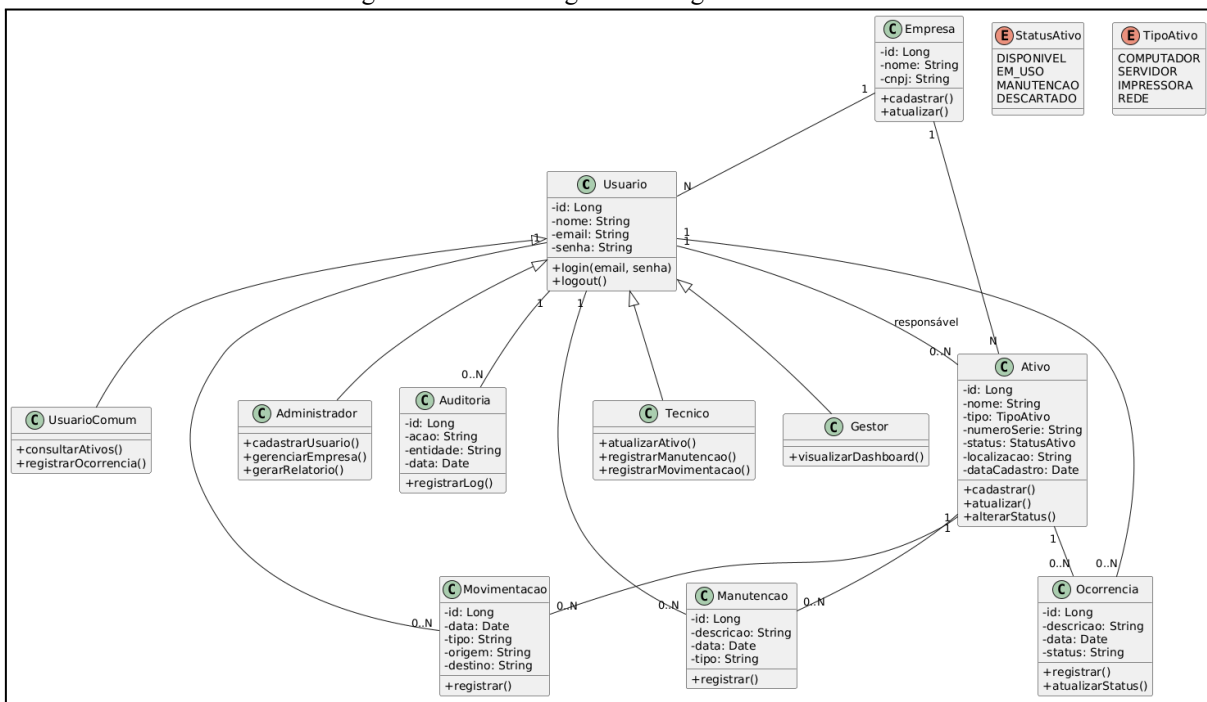


Fonte: Produzido pelo autor.

3.3.2 Diagrama de Classes

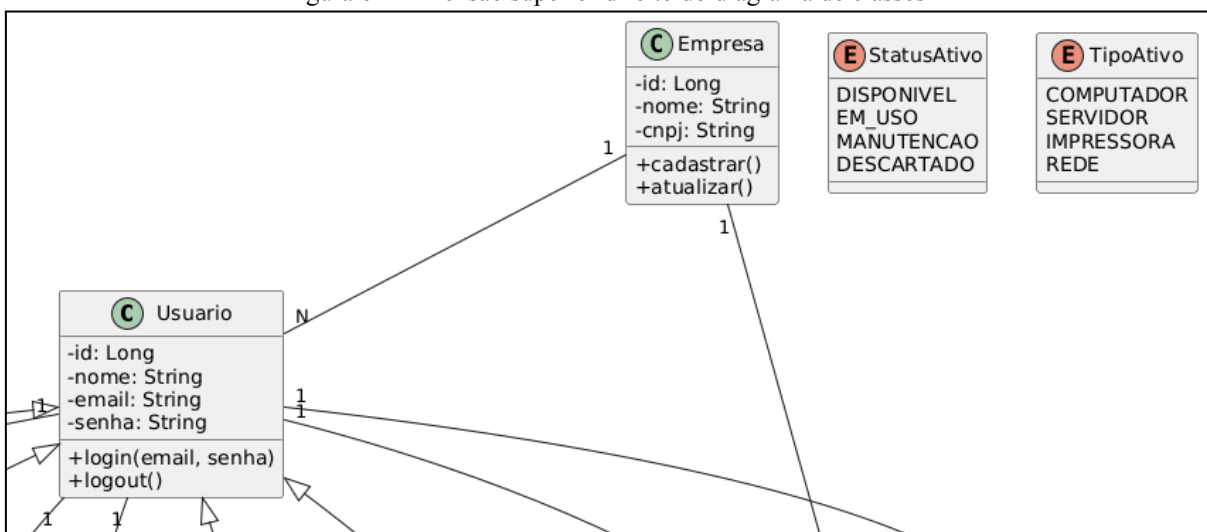
O diagrama de classes apresenta a estrutura do sistema Assetra sob a perspectiva orientada a objetos, incluindo atributos, métodos e níveis de acesso das classes. A modelagem contempla as principais entidades do domínio, bem como suas responsabilidades e interações, garantindo alinhamento com os requisitos funcionais e regras de negócio do sistema.

Figura 5 - Panorama geral do diagrama de classes



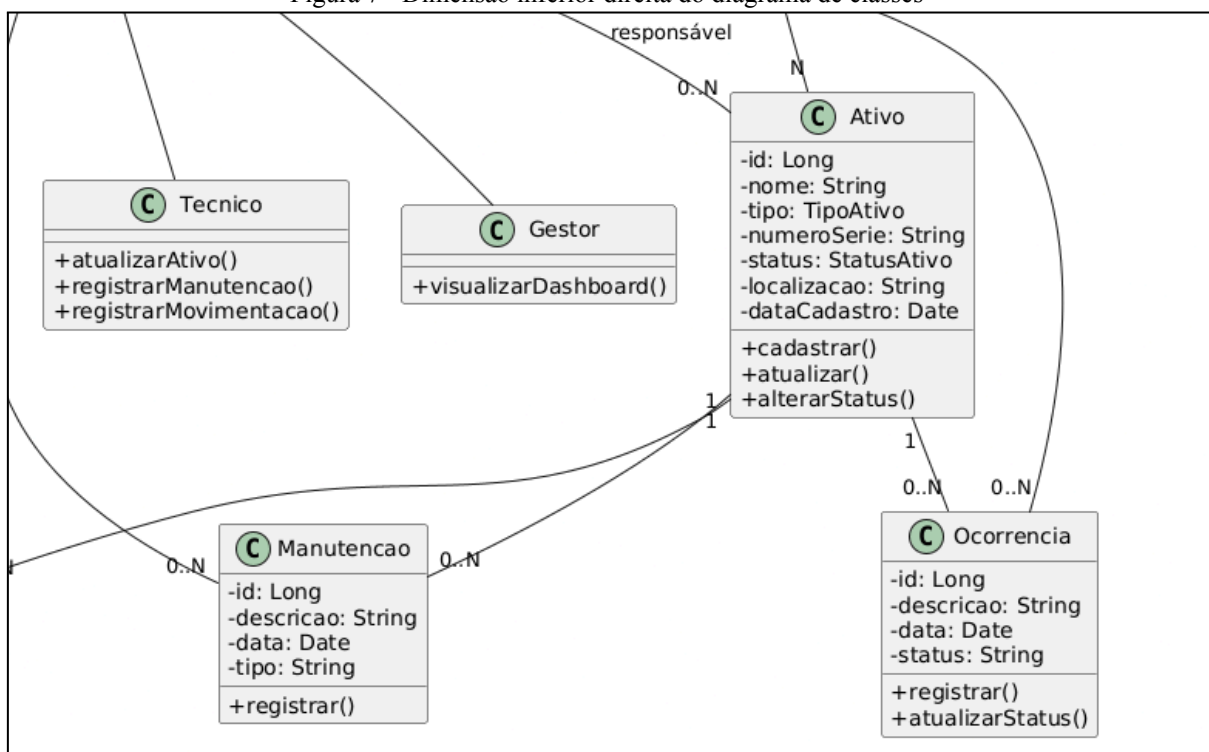
Fonte: Produzido pelo autor.

Figura 6 - Dimensão superior direito do diagrama de classes



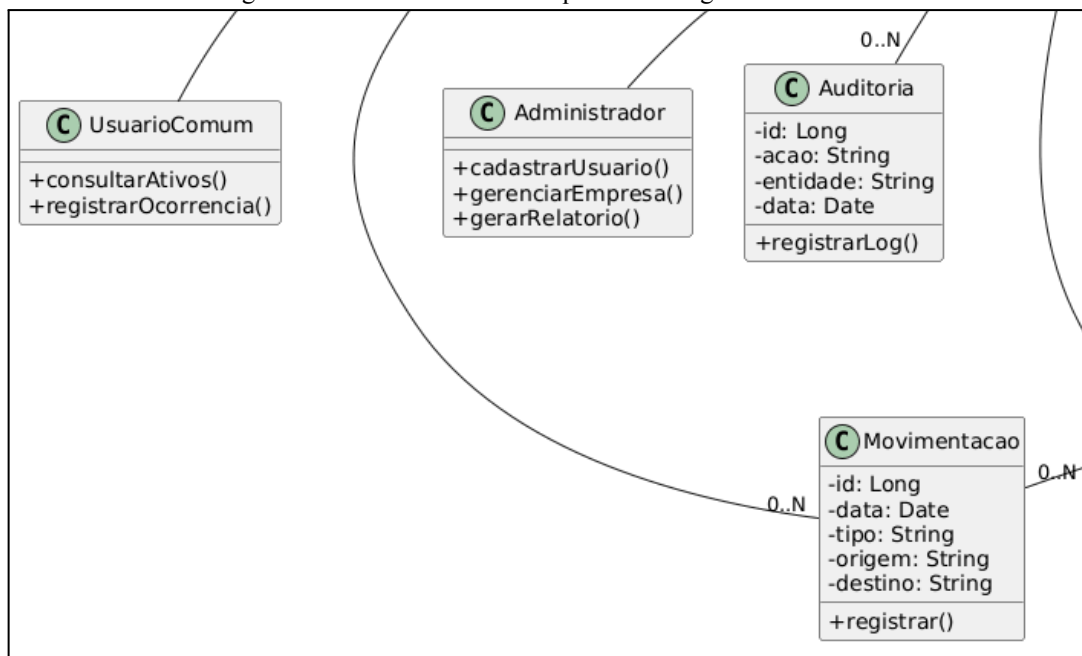
Fonte: Produzido pelo autor.

Figura 7 - Dimensão inferior direita do diagrama de classes



Fonte: Produzido pelo autor.

Figura 8 - Dimensão inferior esquerda do diagrama de classes

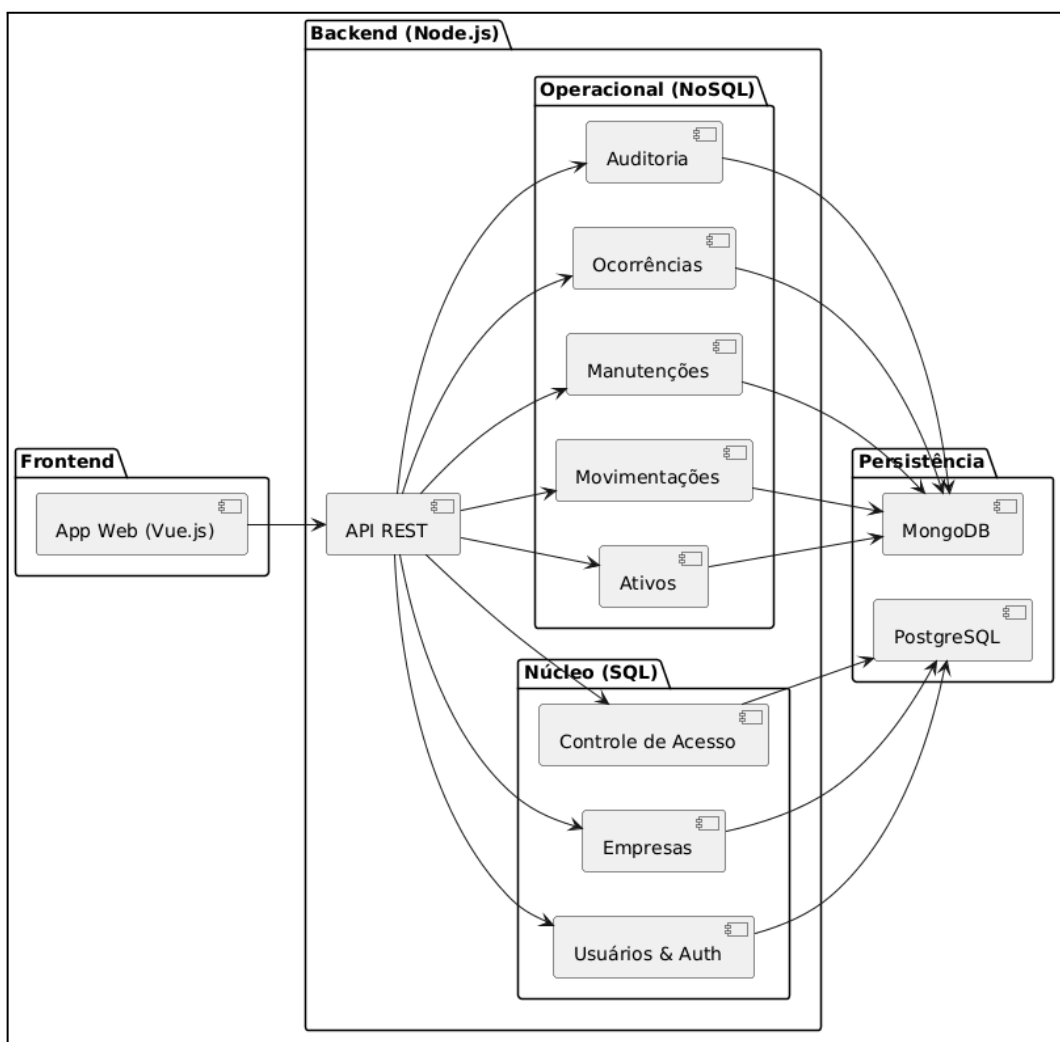


Fonte: Produzido pelo autor.

3.3.3 Diagrama de Componentes

O diagrama de componentes apresenta a organização do sistema Assetra, destacando a separação entre frontend, backend e camada de persistência. O backend, desenvolvido em Node.js, está dividido em dois contextos: o núcleo do sistema, responsável por autenticação, usuários e controle de acesso, persistido em banco relacional (PostgreSQL); e o contexto operacional, responsável pela gestão de ativos e seus históricos, persistido em banco NoSQL (MongoDB). Essa estrutura evidencia a utilização de uma abordagem de persistência poliglota.

Figura 9 - Panorama geral do diagrama de componentes

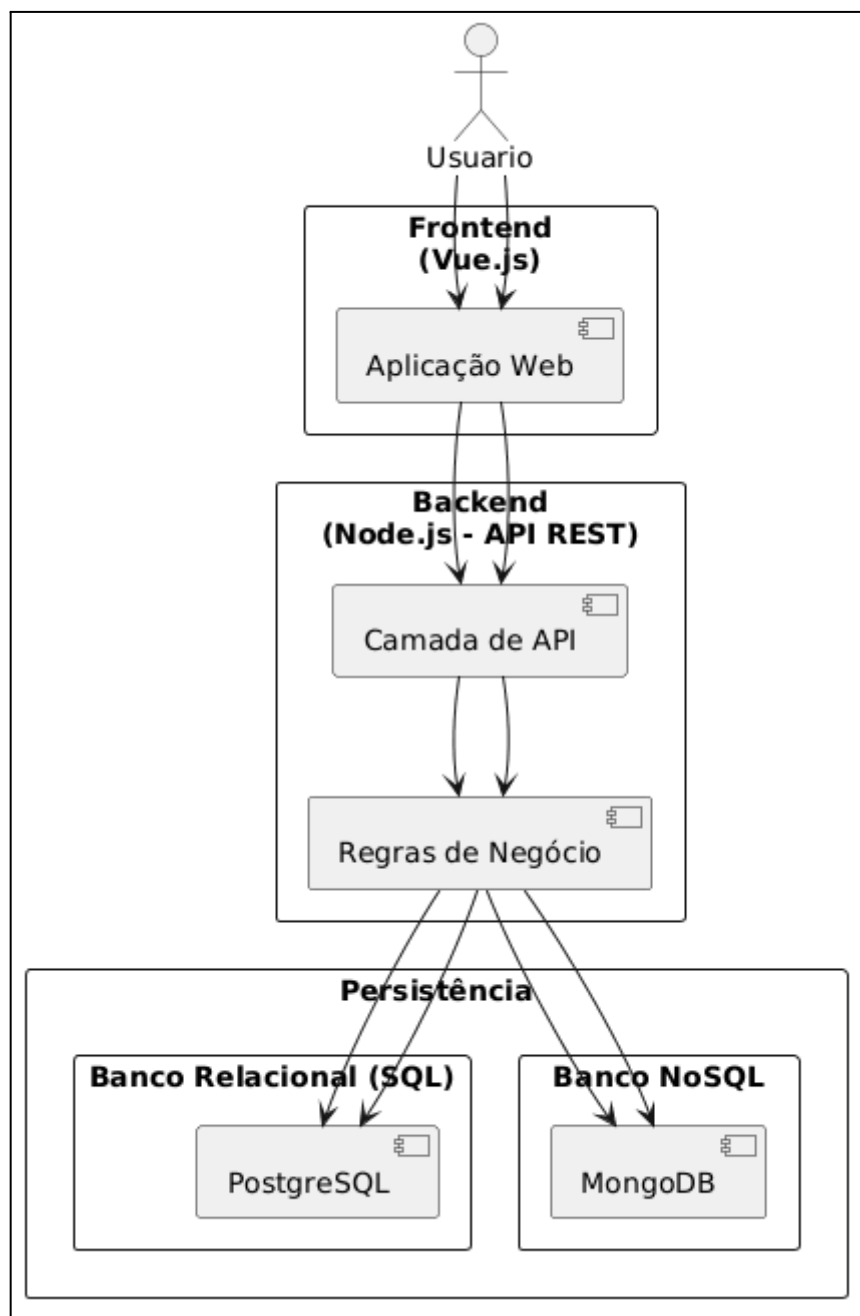


Fonte: Produzido pelo autor.

3.3.4 Diagrama de Arquitetura

O diagrama de arquitetura apresenta a estrutura do sistema Assetra, mostrando a interação entre o usuário, o frontend em Vue.js, o backend em Node.js e a camada de persistência. O sistema utiliza dois bancos de dados: PostgreSQL para dados estruturados e MongoDB para dados operacionais, seguindo uma abordagem de persistência poliglota.

Figura 10 - Panorama geral do diagrama de arquitetura



Fonte: Produzido pelo autor.

3.4 MAPEAMENTO OBJETO-RELACIONAL (ORM)

O sistema utiliza uma abordagem de persistência baseada em ORM (Object-Relational Mapping) para o banco de dados relacional e ODM (Object-Document Mapping) para o banco NoSQL.

No contexto relacional, é utilizado o Prisma ORM para mapear as entidades da aplicação para tabelas no banco PostgreSQL. Cada classe ou modelo definido na aplicação corresponde a uma tabela, enquanto seus atributos representam colunas.

Exemplo de Mapeamento (Prisma)

Classe/Modelo: Usuario

Tabela: usuarios

Atributo	Tipo	Coluna
id	String (UUID)	id
nome	String	nome
email	String	email
senha	String	senha
empresaId	String	empresa_id

Fonte: Produzido pelo autor.

Classe/Modelo: Empresa

Tabela: empresas

Atributo	Tipo	Coluna
id	String	id
nome	String	nome

Fonte: Produzido pelo autor.

Exemplo de Mapeamento (MongoDB)

Modelo: Ativo

Coleção: ativos

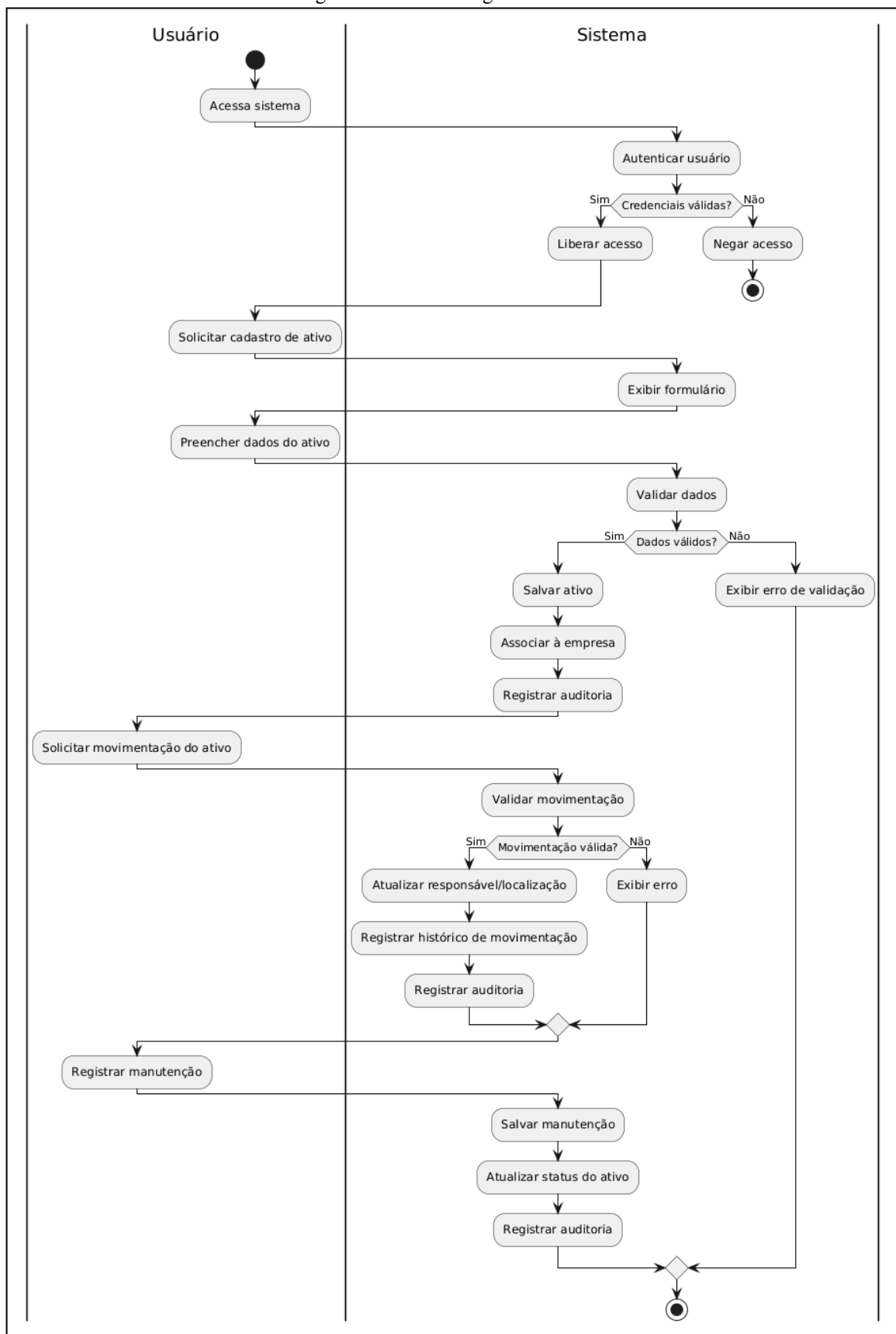
Campo	Tipo	Descrição
_id	ObjectId	Identificador do documento
nome	String	Nome do ativo
tipo	String	Tipo do ativo
numeroSerie	String	Número de série
status	String	Estado do ativo
historico	Array	Lista de eventos do ativo

Fonte: Produzido pelo autor.

3.5 BPMN (BUSINESS PROCESS MODEL AND NOTATION)

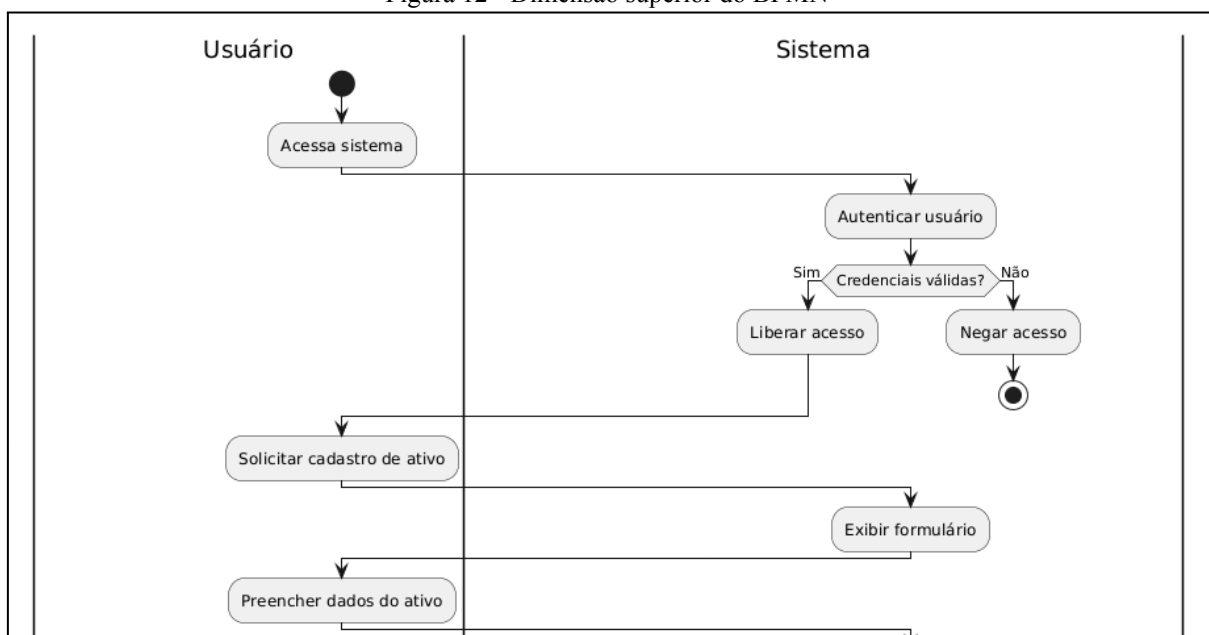
O diagrama BPMN representa o fluxo completo de gestão de ativos no sistema Assetra, incluindo autenticação, cadastro, validação, movimentação e registro de manutenção. O processo também contempla regras de negócio como validação de dados, associação do ativo à empresa, registro automático de movimentações e auditoria das ações realizadas.

Figura 11 - Panorama geral do BPMN



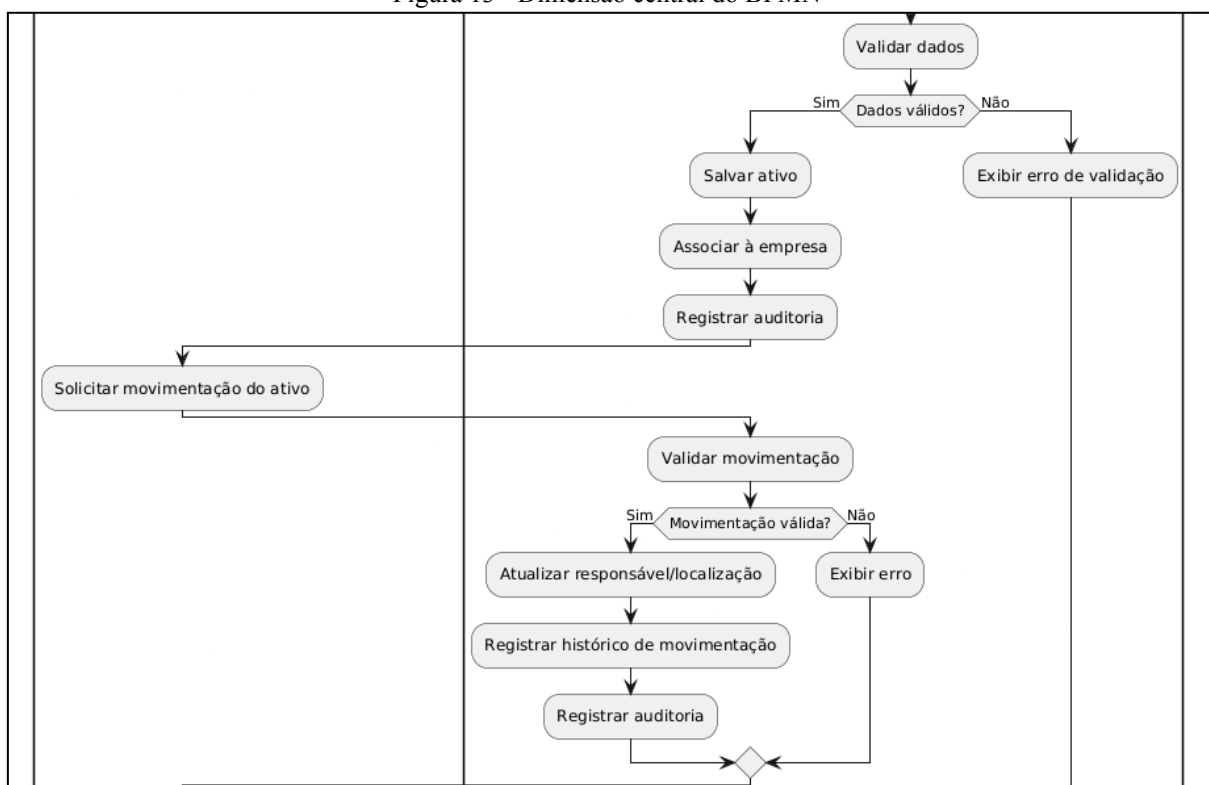
Fonte: Produzido pelo autor.

Figura 12 - Dimensão superior do BPMN



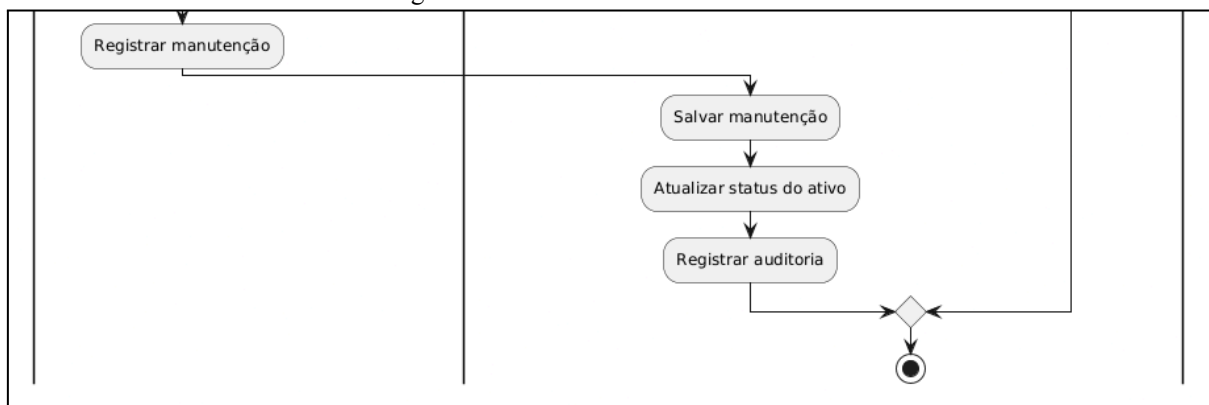
Fonte: Produzido pelo autor.

Figura 13 - Dimensão central do BPMN



Fonte: Produzido pelo autor.

Figura 14 - Dimensão inferior do BPMN



Fonte: Produzido pelo autor.

DEI

4 DOCUMENTO DE ESPECIFICAÇÃO DE INTERFACES (DEI)

4.1 WIREFRAMES

Os wireframes foram desenvolvidos para representar a estrutura das principais telas do sistema, focando na disposição dos elementos e na navegação, sem considerar aspectos visuais detalhados.

Figura 16 - Wireframe da tela de login

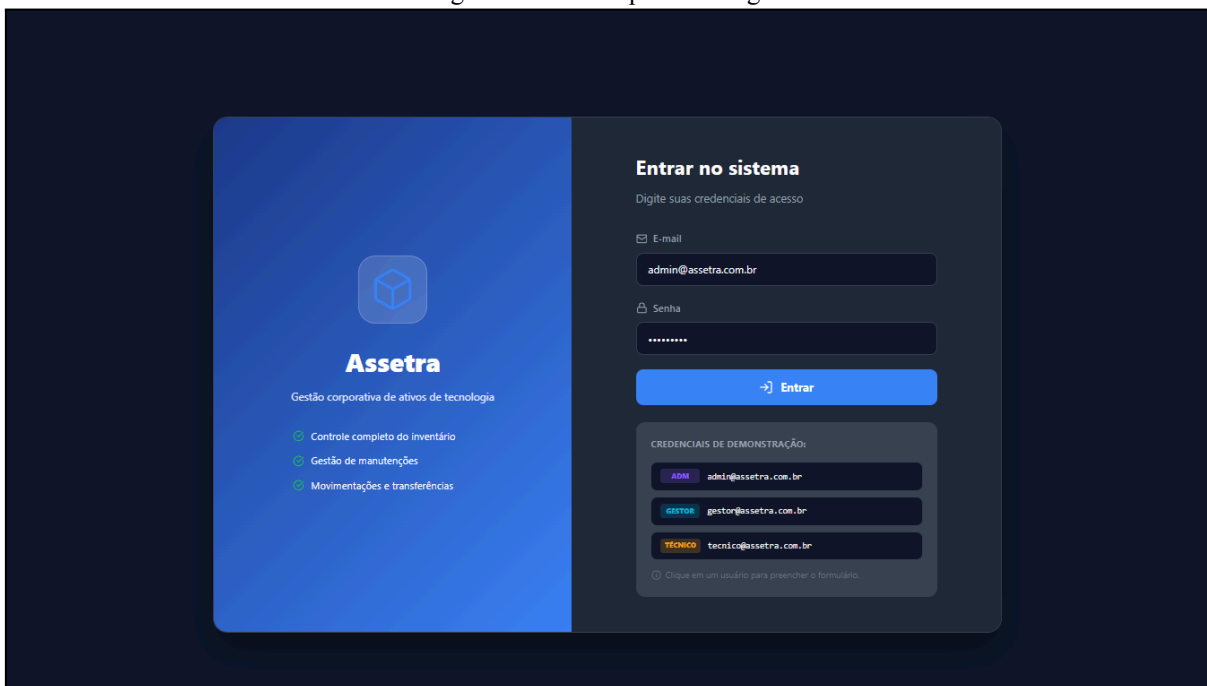
Figura 17 - Wireframe da dashboard

Fonte: Produzido pelo autor.

4.2 MOCKUPS

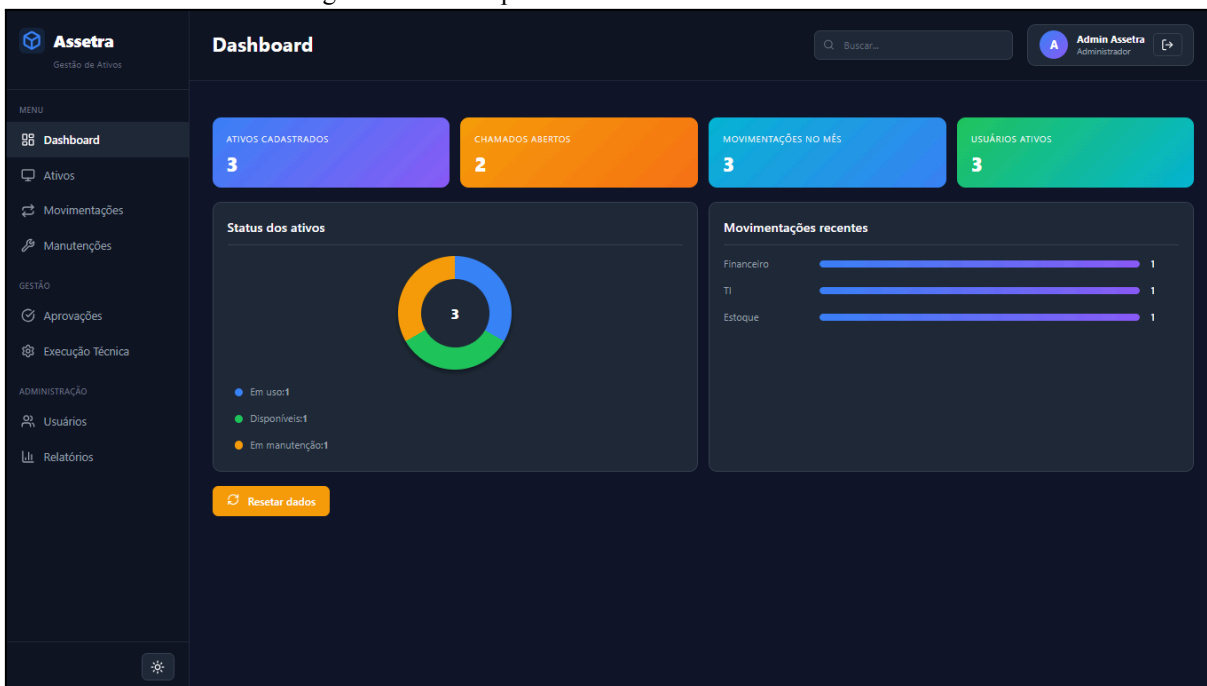
Os mockups foram elaborados com base nas telas já implementadas no sistema, sendo organizados na ferramenta Figma para fins de documentação e apresentação da interface.

Figura 18 - Mockup tela de login



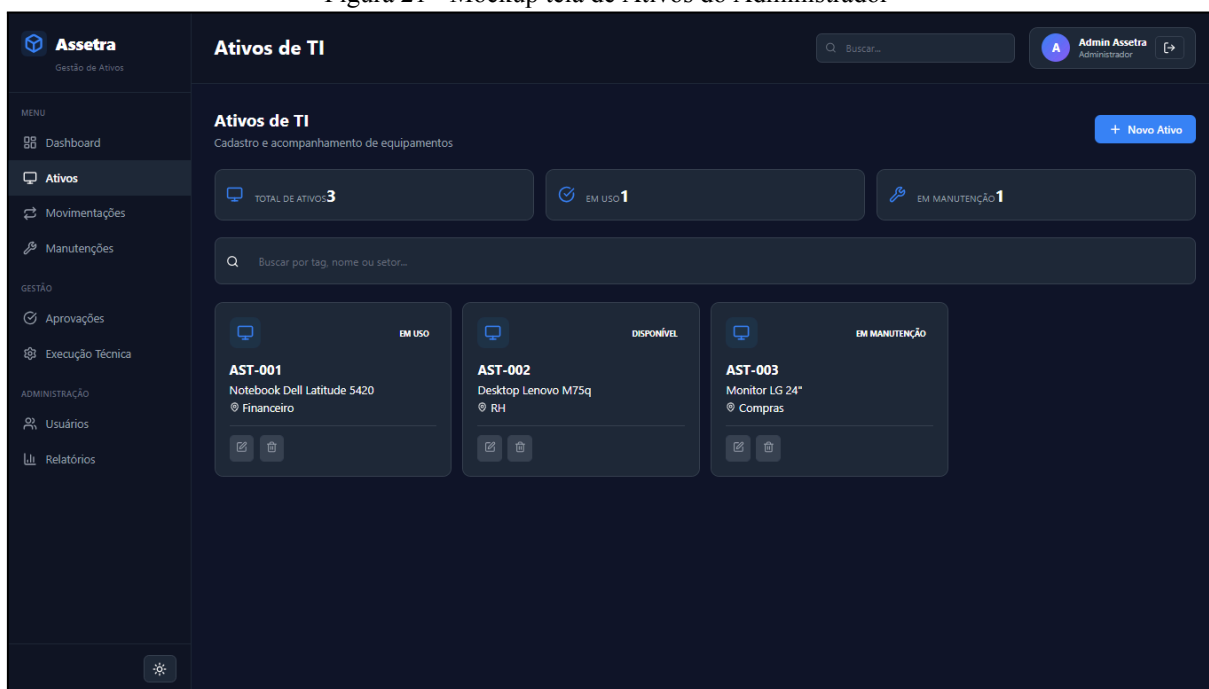
Fonte: Produzido pelo autor.

Figura 19 - Mockup tela de Dashboard do Administrador



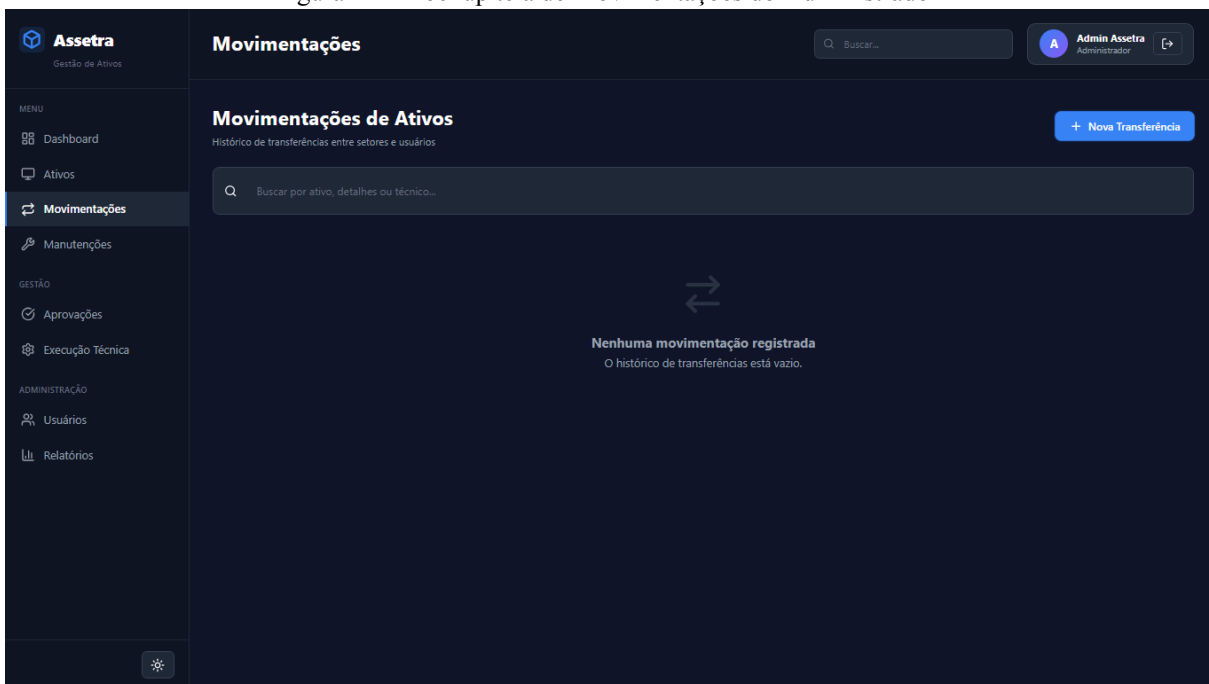
Fonte: Produzido pelo autor.

Figura 21 - Mockup tela de Ativos do Administrador



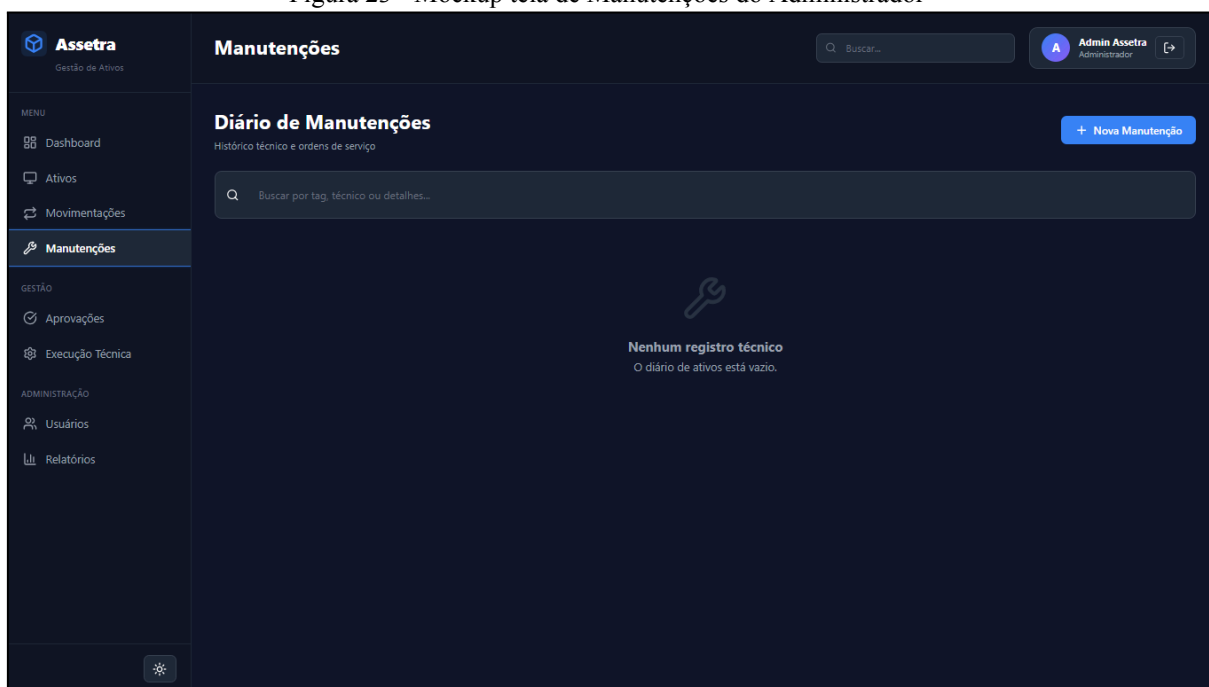
Fonte: Produzido pelo autor.

Figura 22 - Mockup tela de Movimentações do Administrador



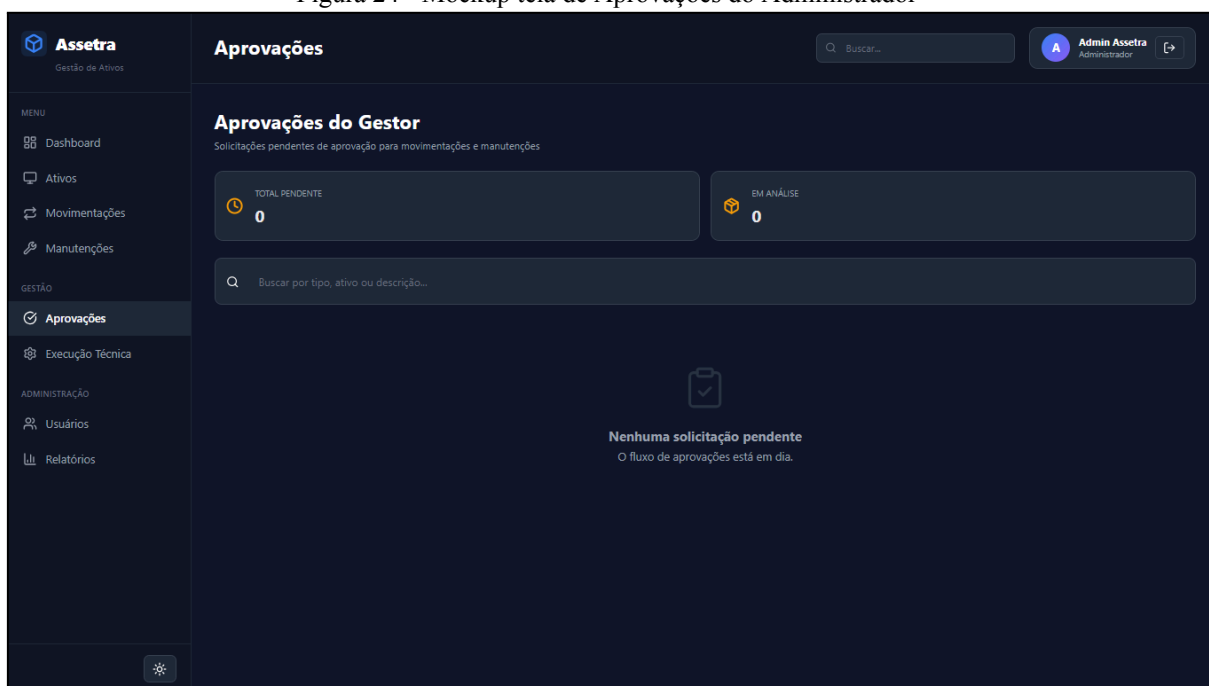
Fonte: Produzido pelo autor.

Figura 23 - Mockup tela de Manutenções do Administrador



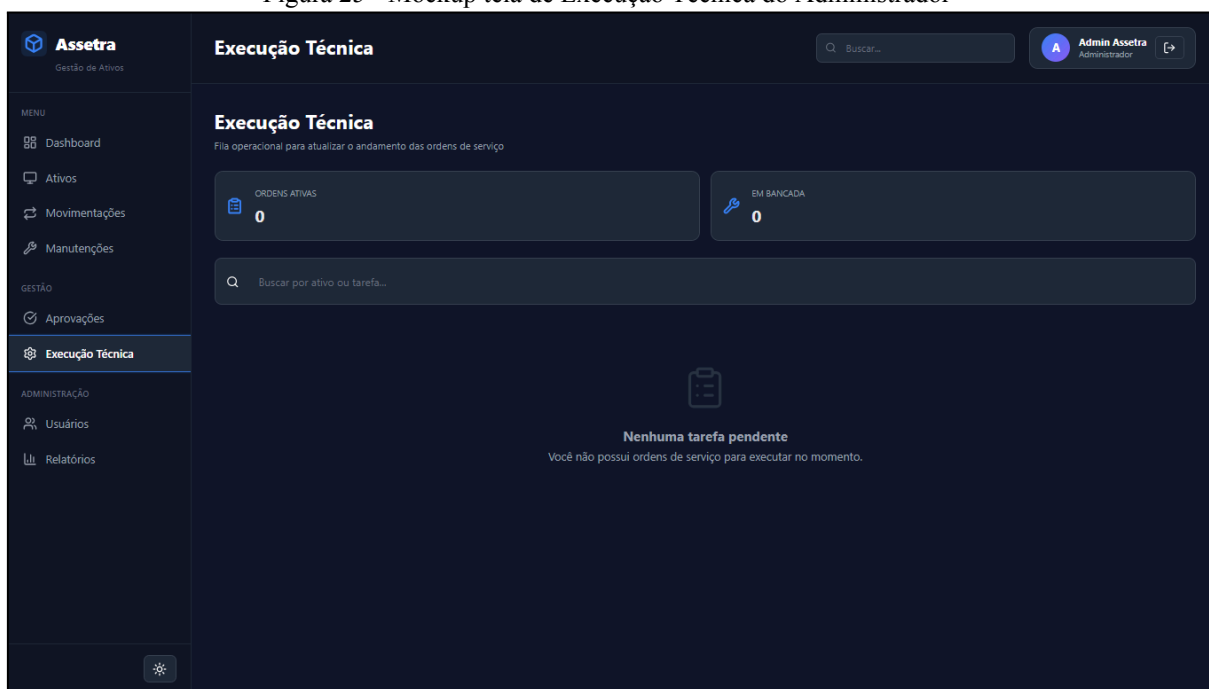
Fonte: Produzido pelo autor.

Figura 24 - Mockup tela de Aprovações do Administrador



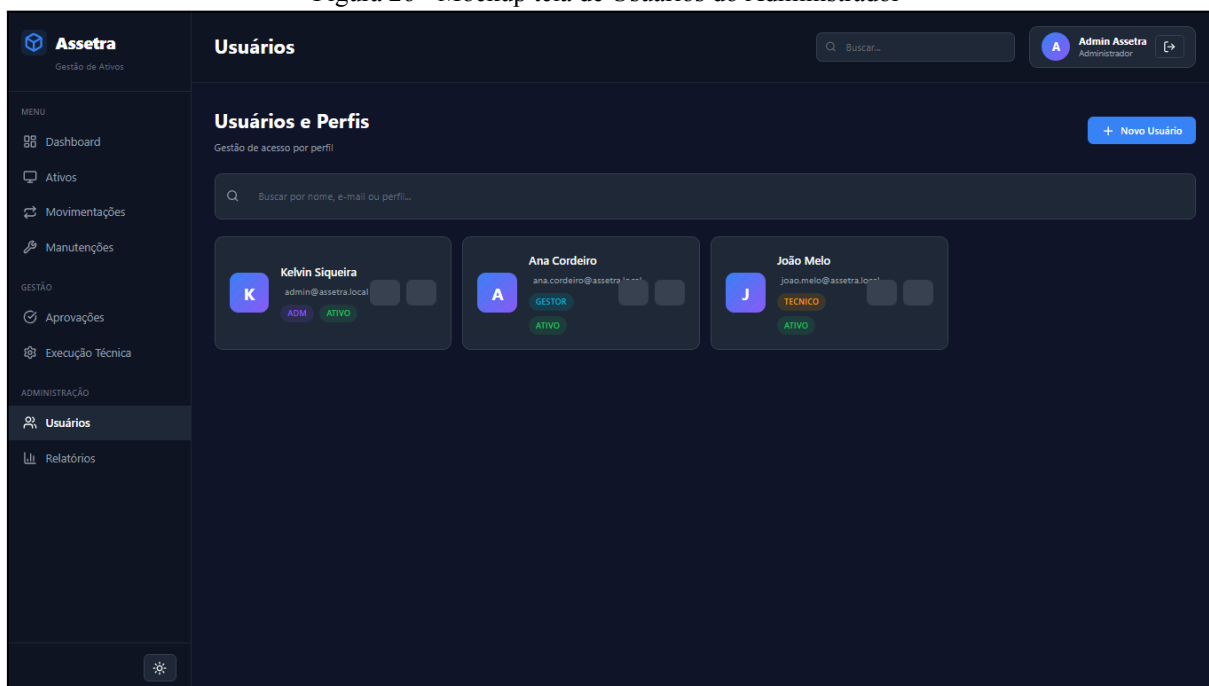
Fonte: Produzido pelo autor.

Figura 25 - Mockup tela de Execução Técnica do Administrador



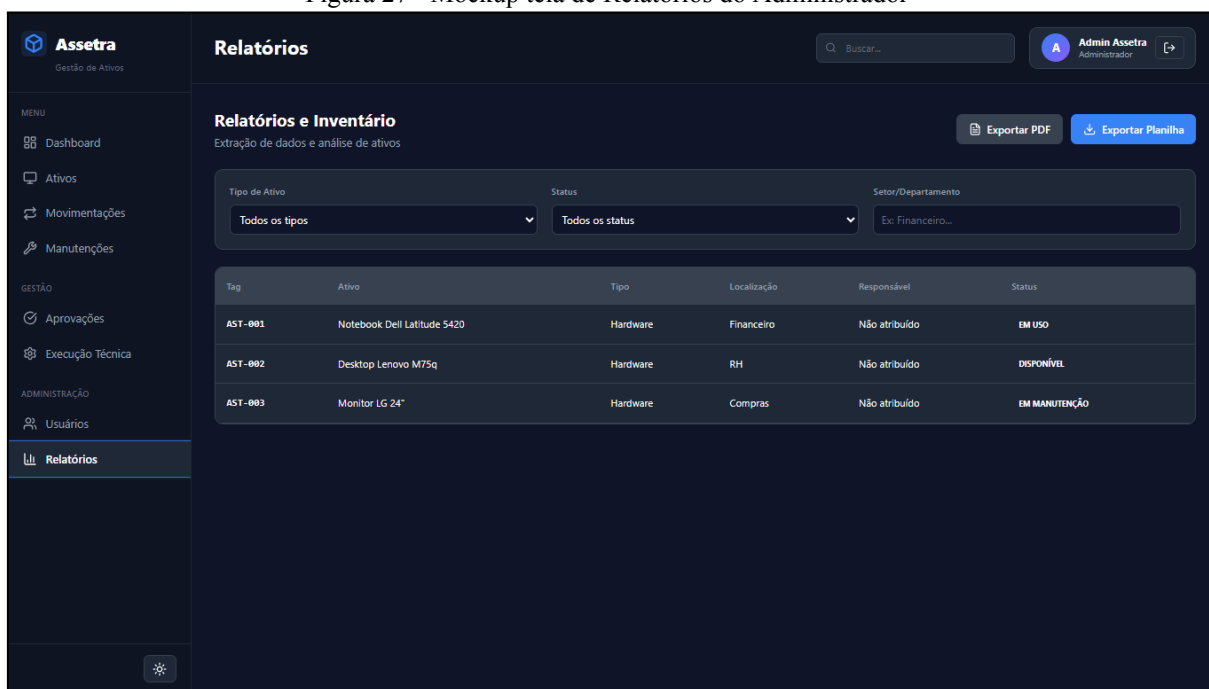
Fonte: Produzido pelo autor.

Figura 26 - Mockup tela de Usuários do Administrador



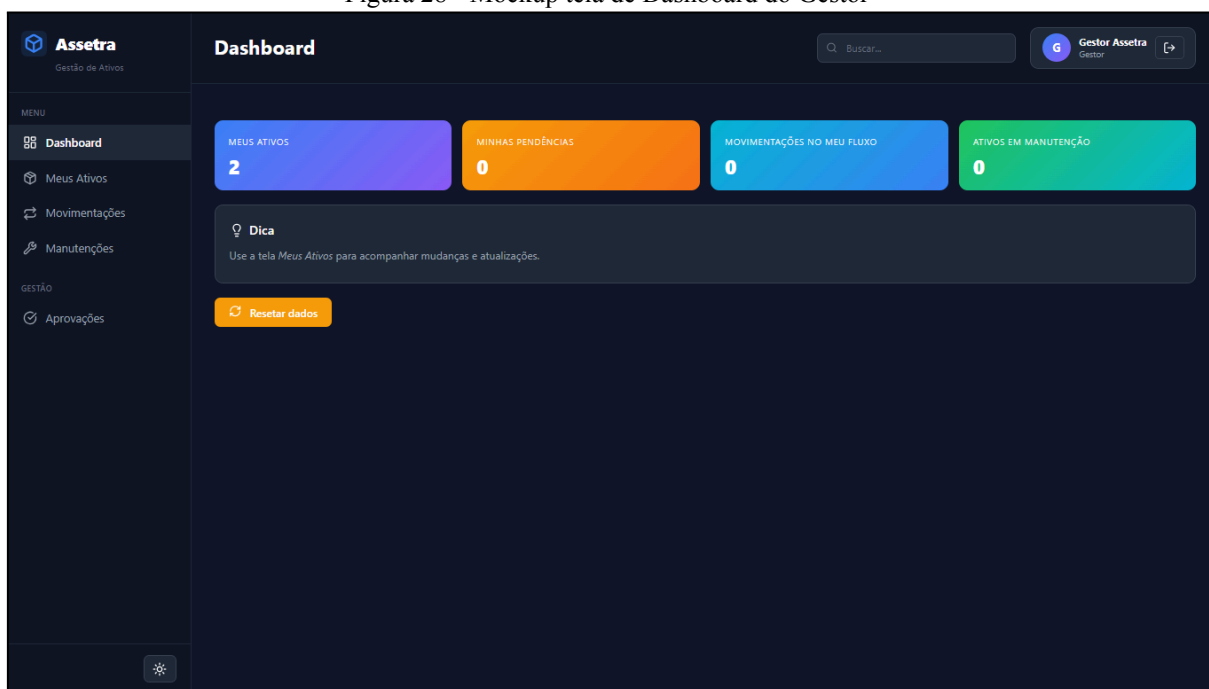
Fonte: Produzido pelo autor.

Figura 27 - Mockup tela de Relatórios do Administrador



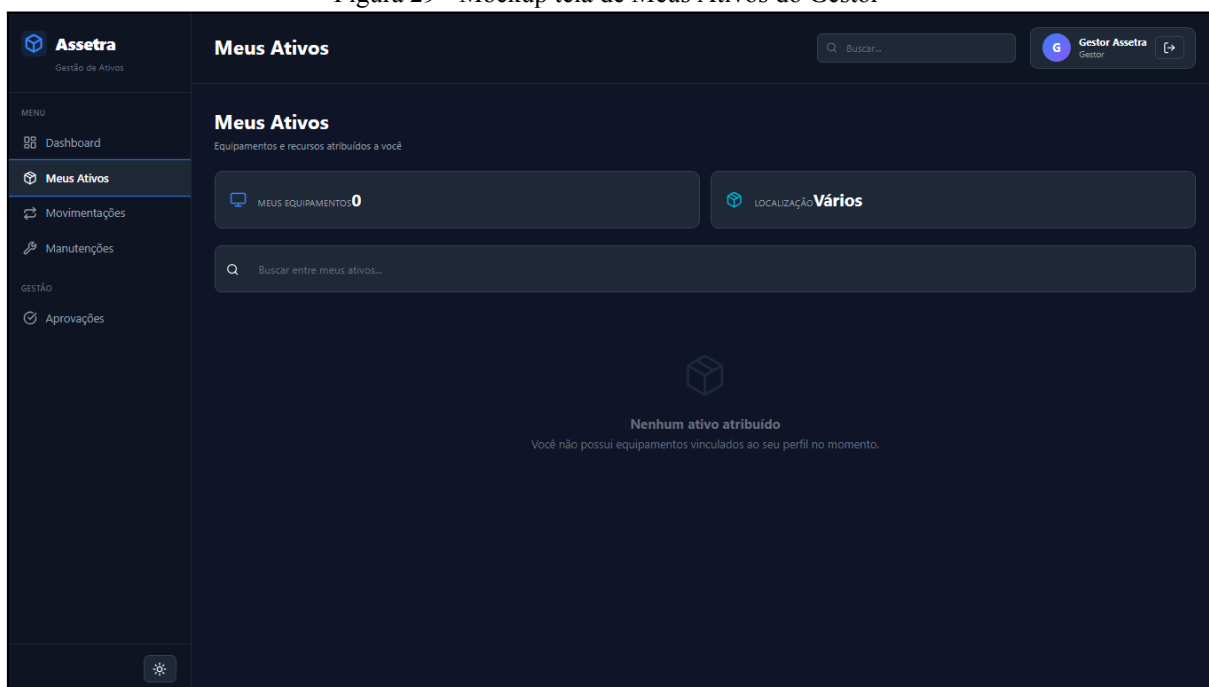
Fonte: Produzido pelo autor.

Figura 28 - Mockup tela de Dashboard do Gestor



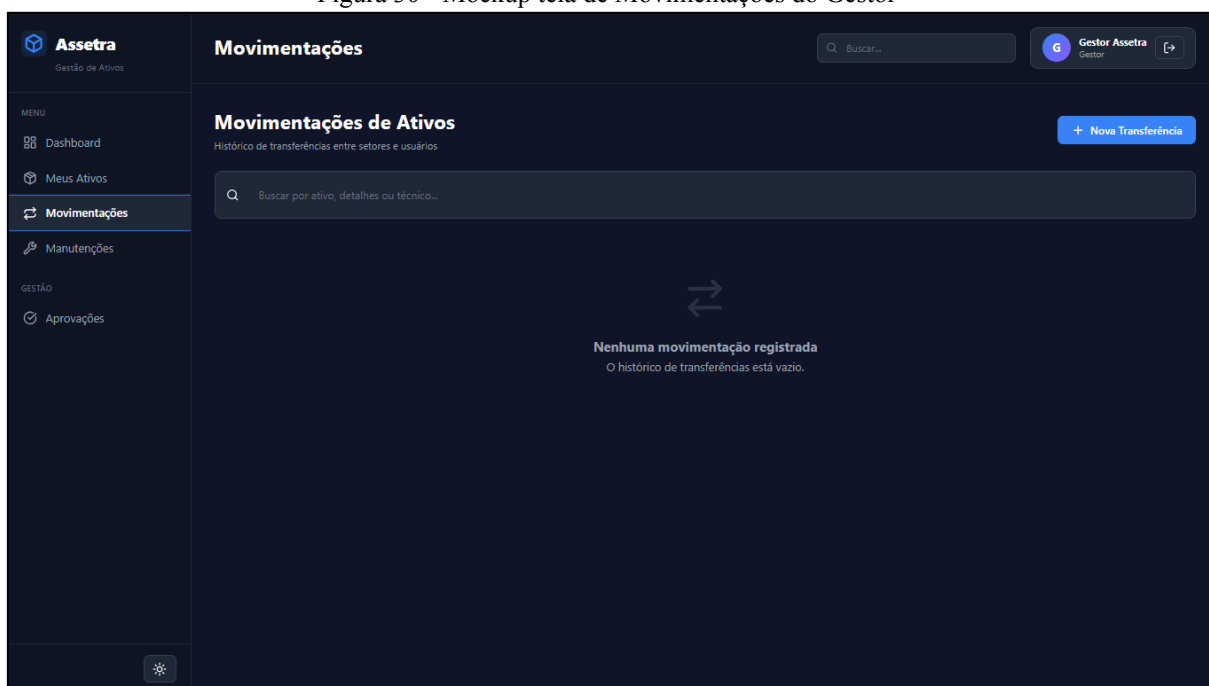
Fonte: Produzido pelo autor.

Figura 29 - Mockup tela de Meus Ativos do Gestor



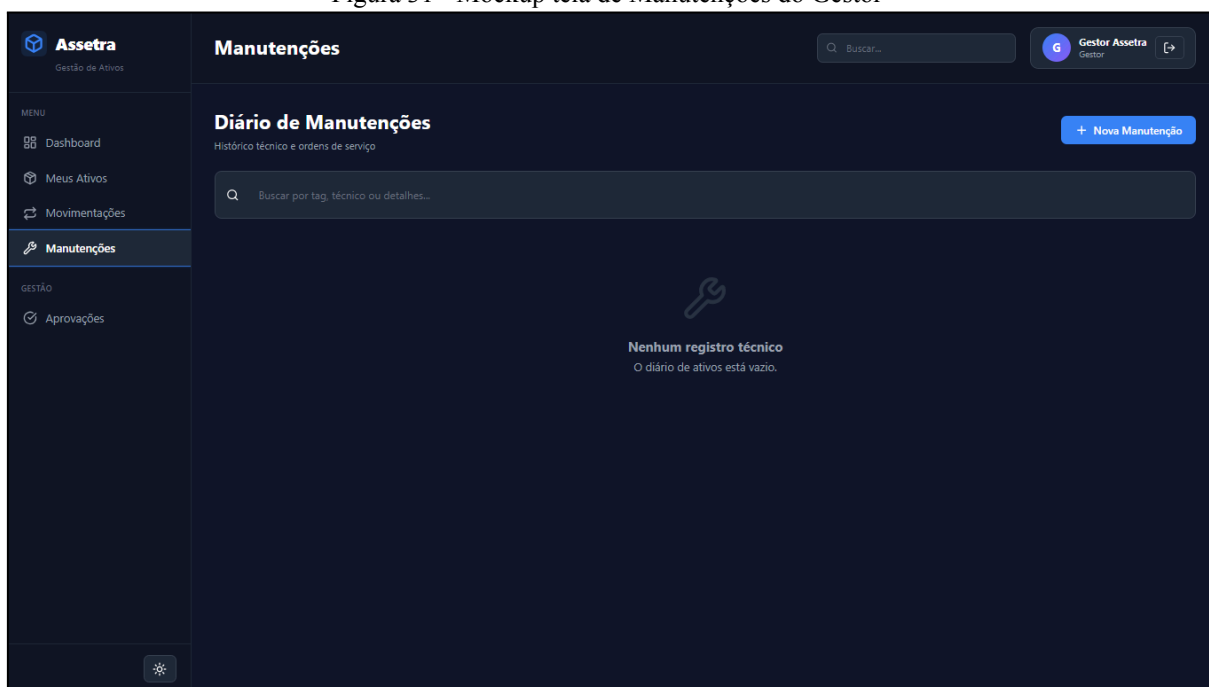
Fonte: Produzido pelo autor.

Figura 30 - Mockup tela de Movimentações do Gestor



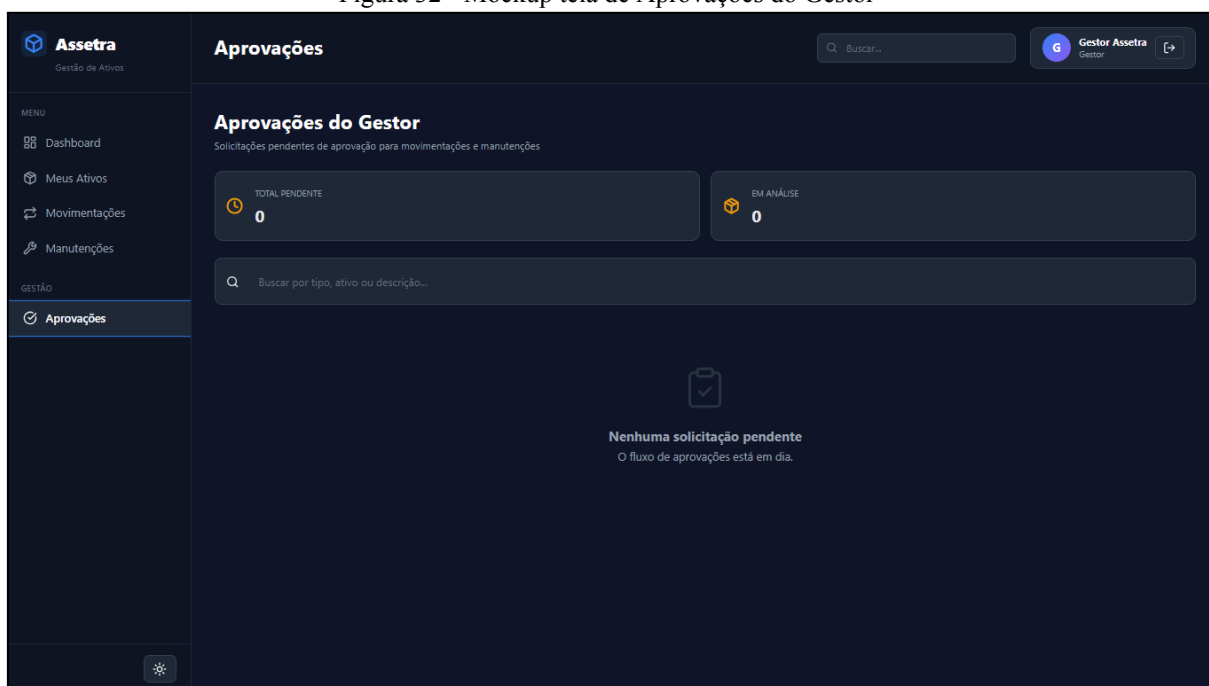
Fonte: Produzido pelo autor.

Figura 31 - Mockup tela de Manutenções do Gestor



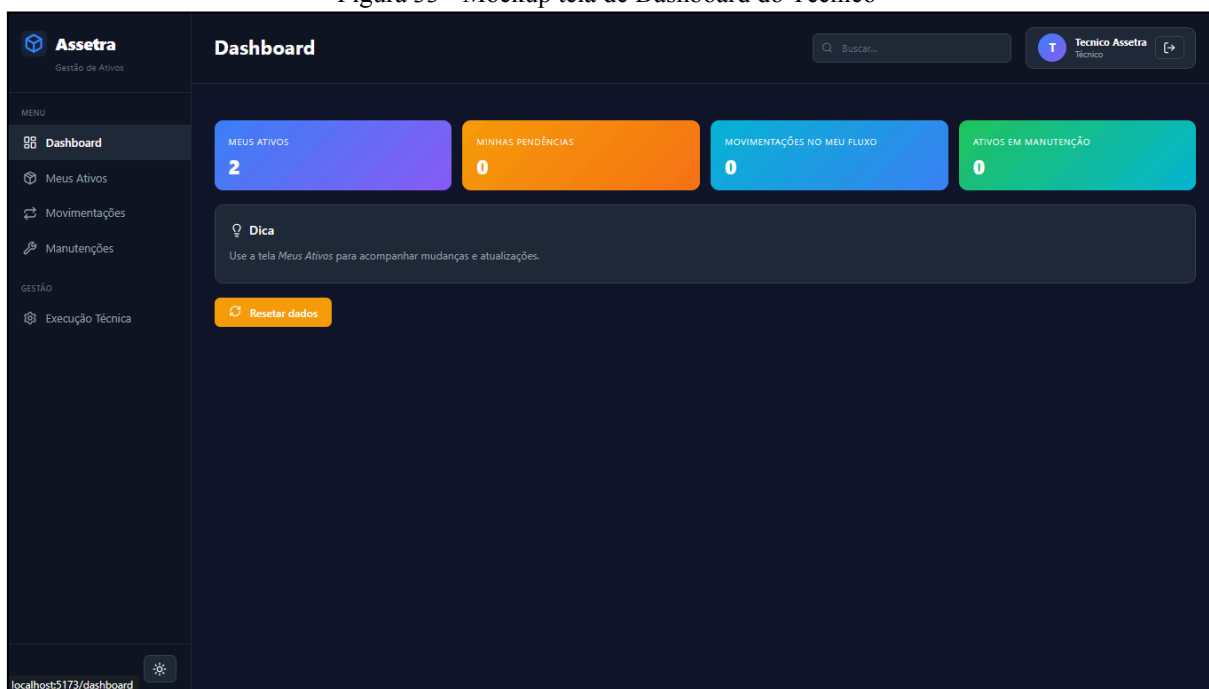
Fonte: Produzido pelo autor.

Figura 32 - Mockup tela de Aprovações do Gestor



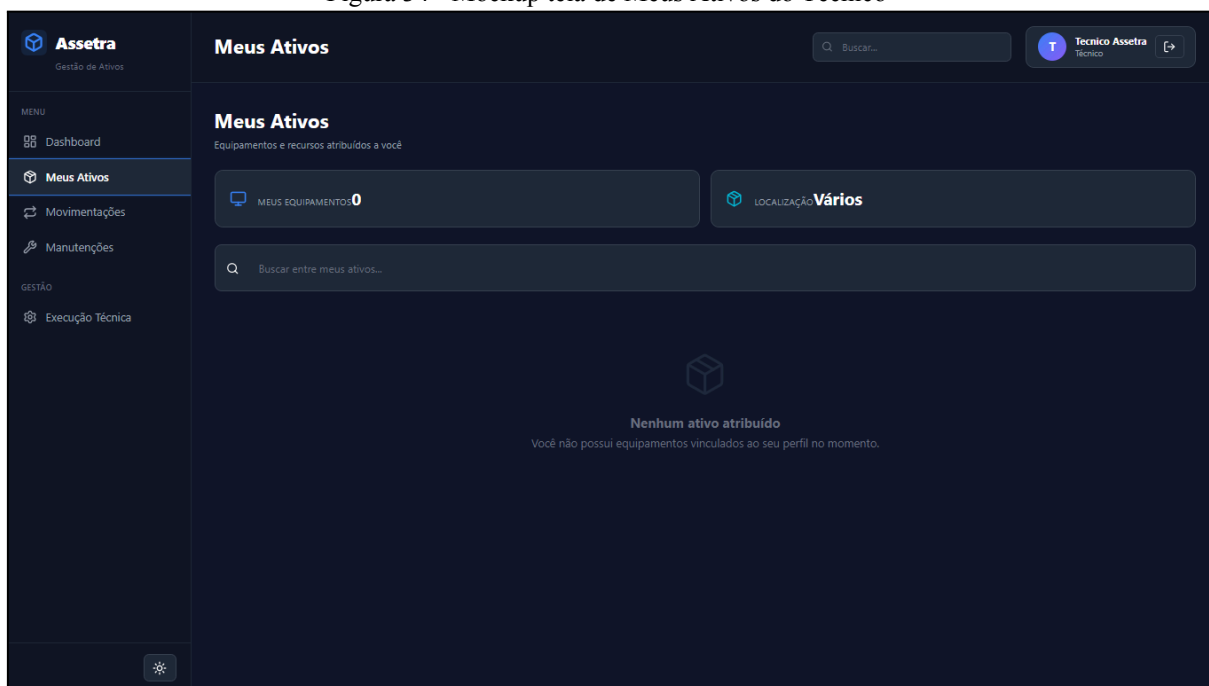
Fonte: Produzido pelo autor.

Figura 33 - Mockup tela de Dashboard do Técnico



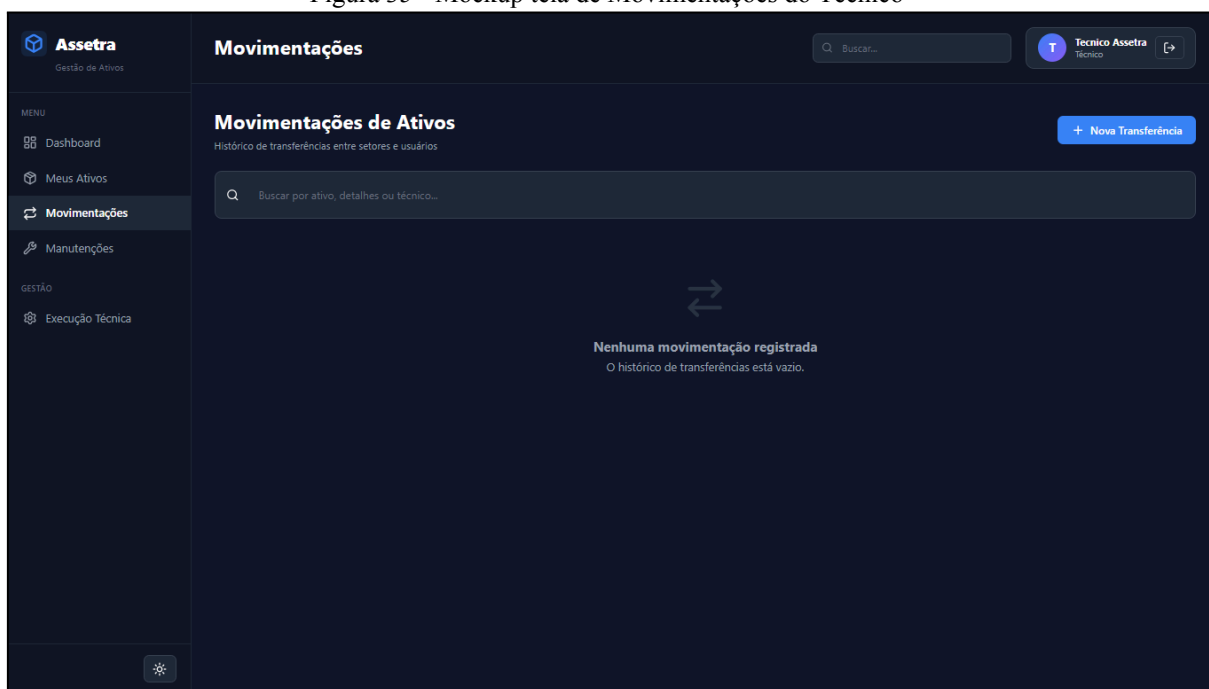
Fonte: Produzido pelo autor.

Figura 34 - Mockup tela de Meus Ativos do Técnico



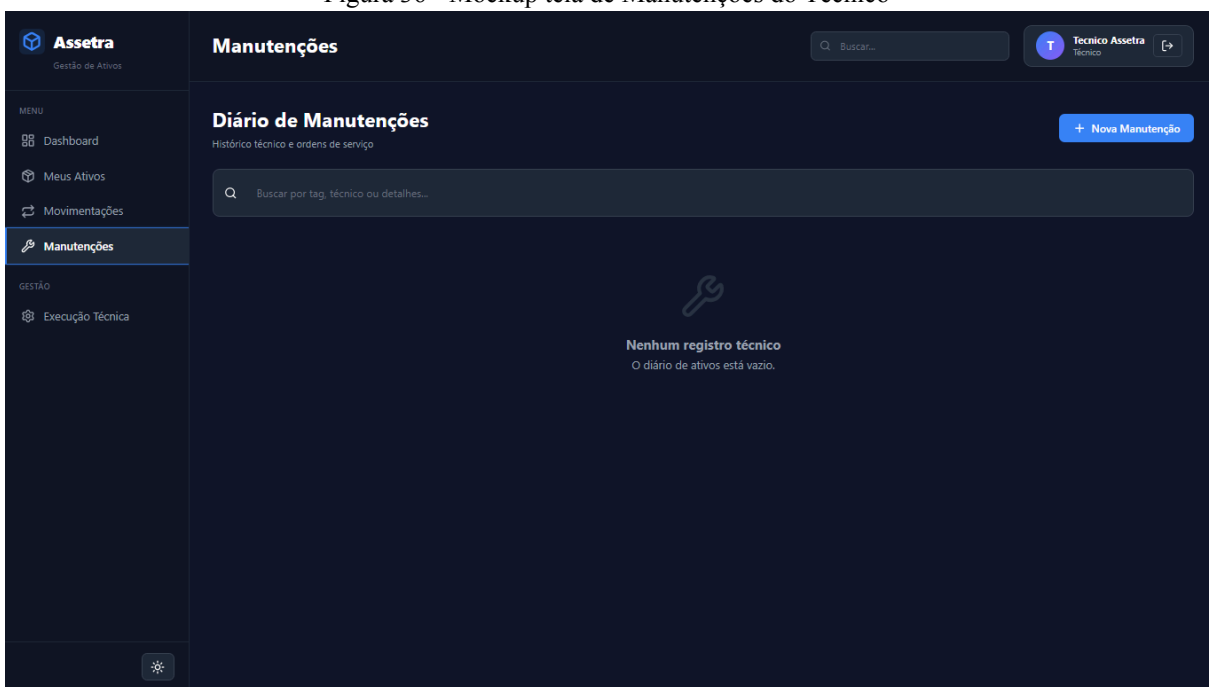
Fonte: Produzido pelo autor.

Figura 35 - Mockup tela de Movimentações do Técnico



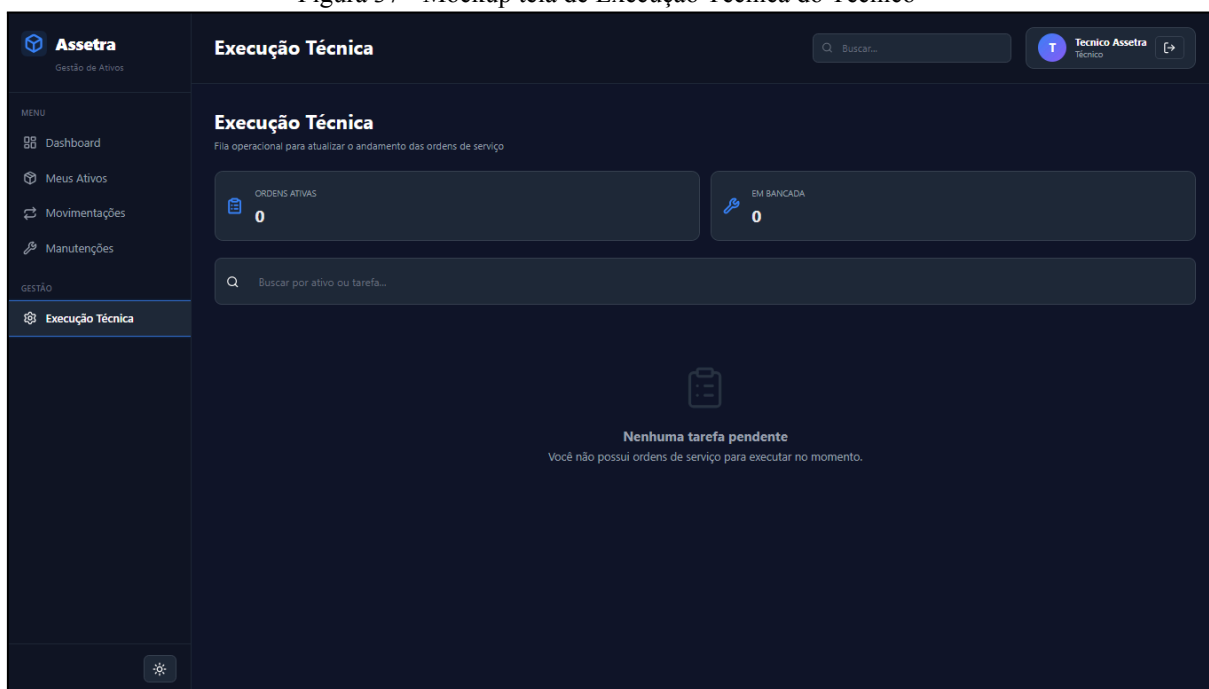
Fonte: Produzido pelo autor.

Figura 36 - Mockup tela de Manutenções do Técnico



Fonte: Produzido pelo autor.

Figura 37 - Mockup tela de Execução Técnica do Técnico

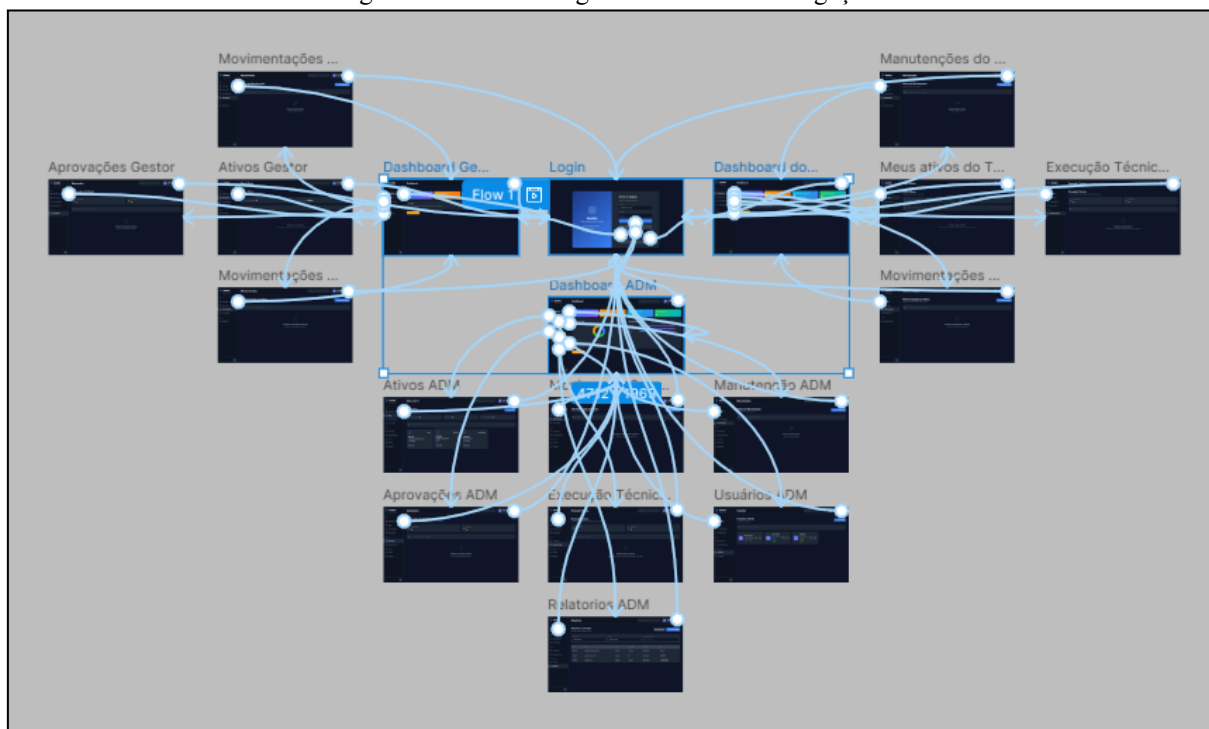


Fonte: Produzido pelo autor.

4.3 FLUXO DE NAVEGAÇÃO

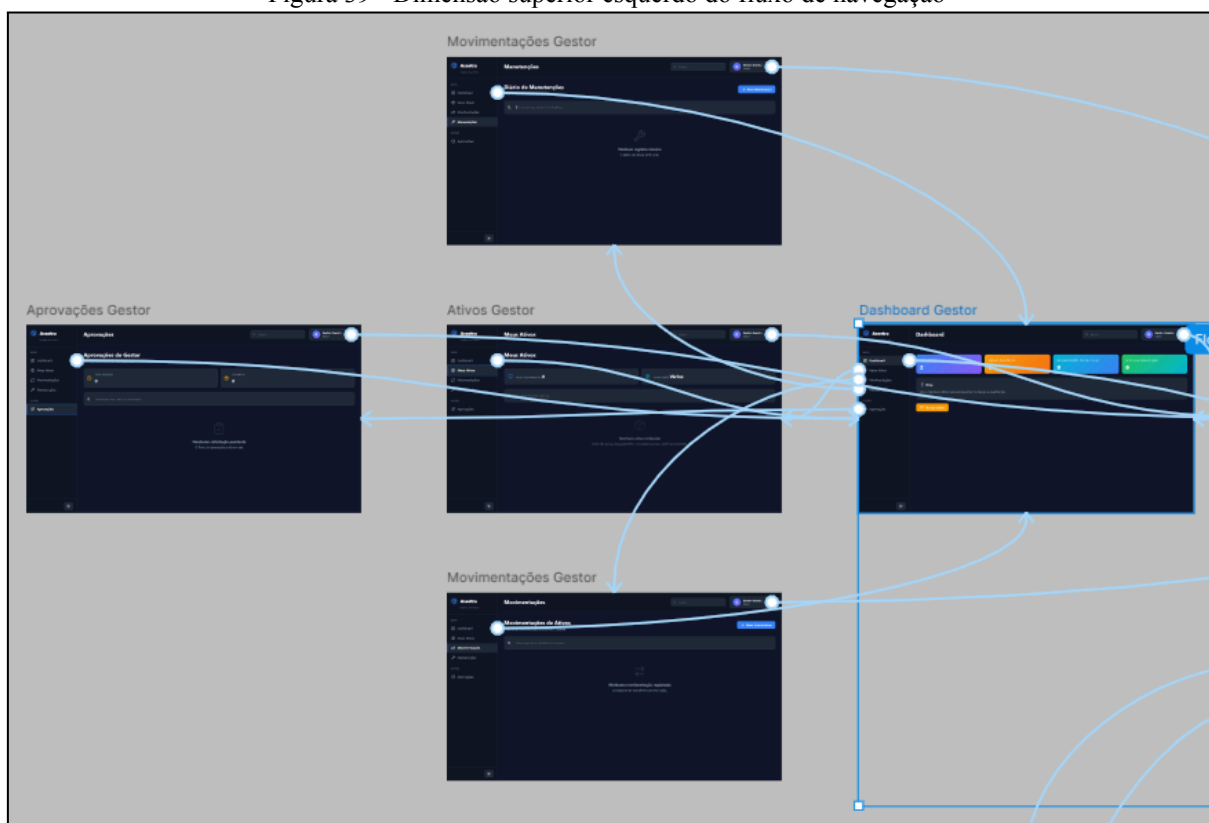
O fluxo de navegação foi representado com base nas telas desenvolvidas, demonstrando o caminho percorrido pelo usuário entre as principais funcionalidades do sistema.

Figura 38 - Panorama geral do fluxo de navegação



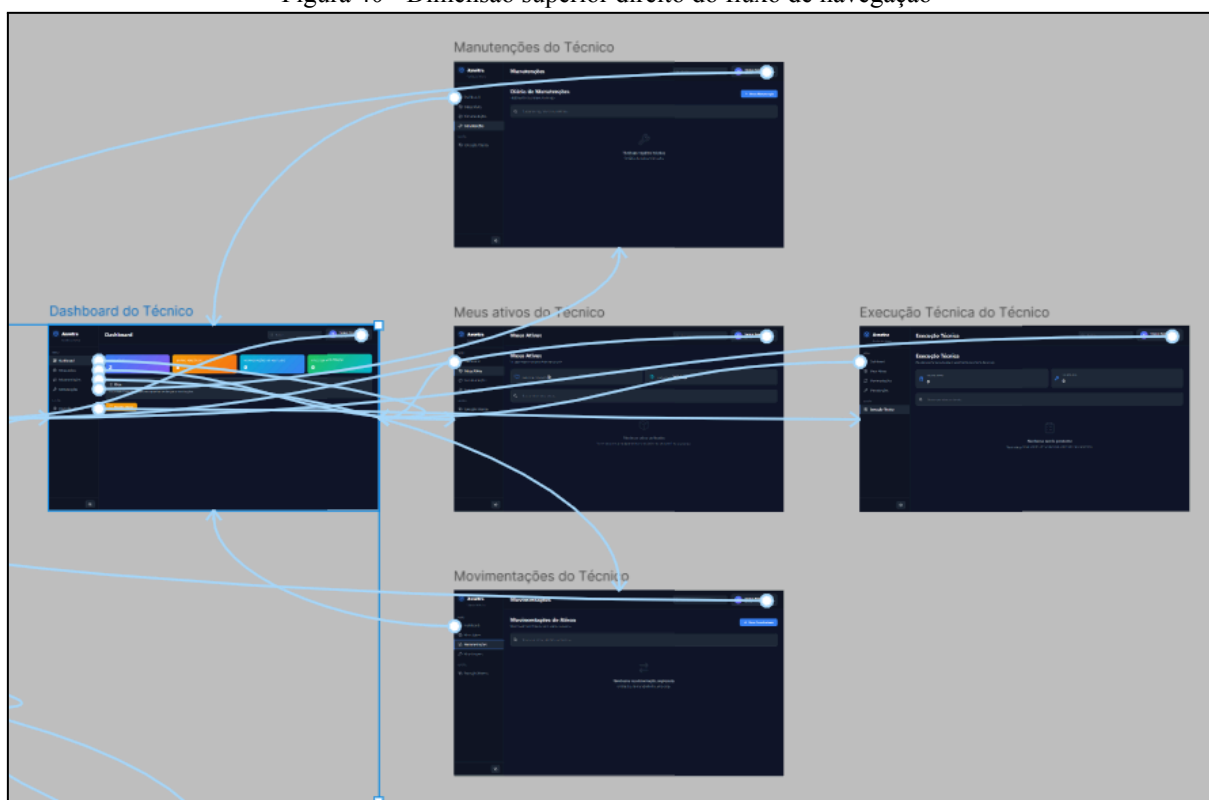
Fonte: Produzido pelo autor.

Figura 39 - Dimensão superior esquerdo do fluxo de navegação



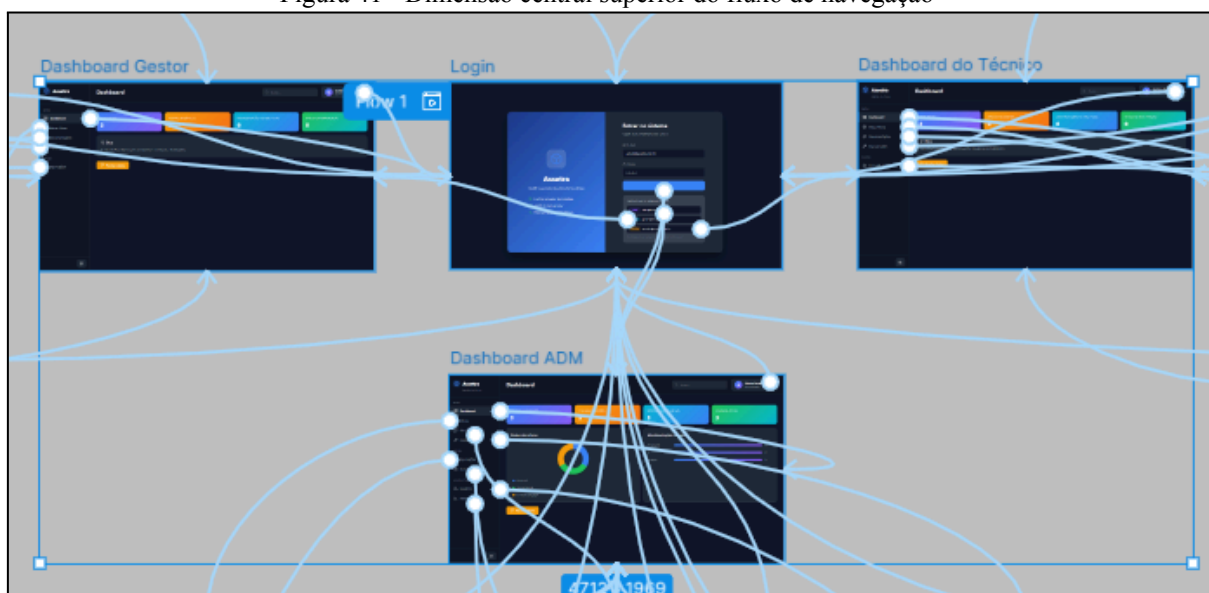
Fonte: Produzido pelo autor.

Figura 40 - Dimensão superior direita do fluxo de navegação



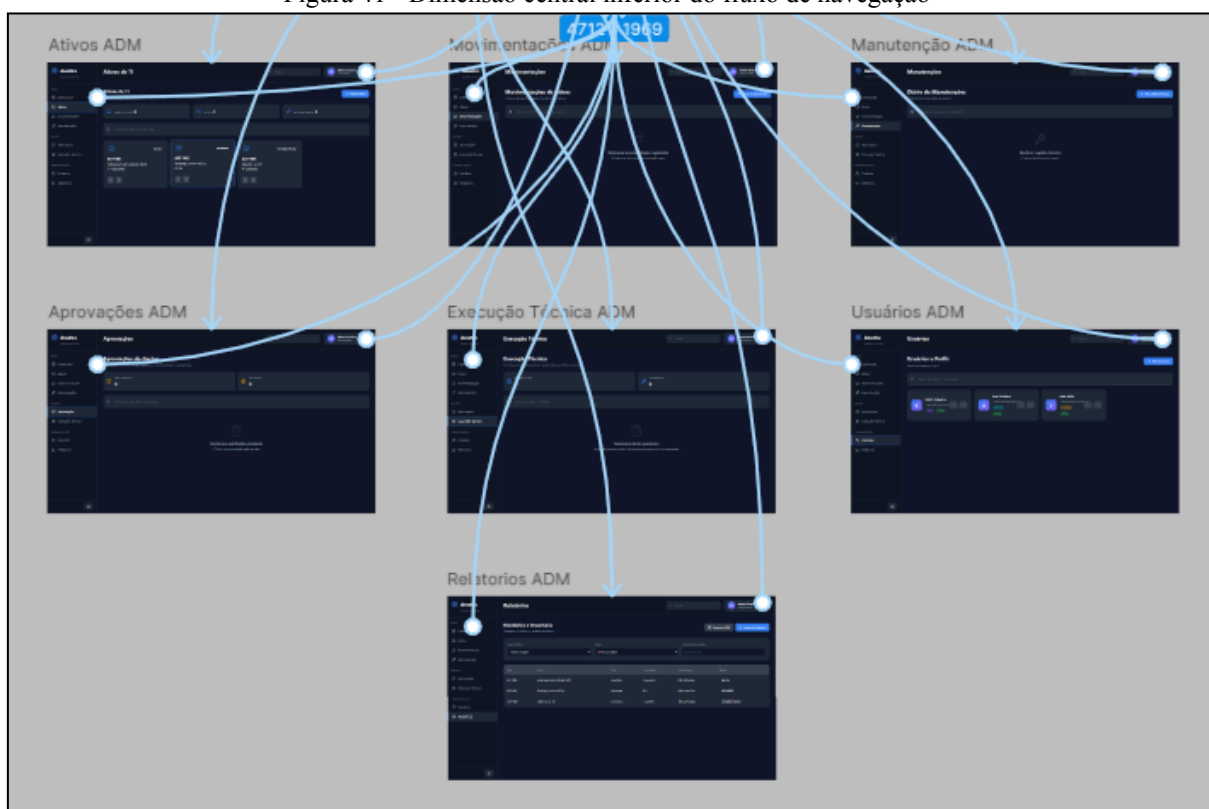
Fonte: Produzido pelo autor.

Figura 41 - Dimensão central superior do fluxo de navegação



Fonte: Produzido pelo autor.

Figura 41 - Dimensão central inferior do fluxo de navegação



Fonte: Produzido pelo autor.

Documentação Técnica

5 DOCUMENTAÇÃO TÉCNICA

5.1 ARQUITETURA DO SISTEMA

Macroestrutura: O Assetra é uma aplicação web cliente-servidor:

Cliente: SPA (Single Page Application) em Vue 3, executada no navegador, que consome a API via HTTPS em produção ou via proxy em desenvolvimento (`VITE_API_BASE_URL / /api`).

Servidor: monólito modular em camadas implementado em Node.js com Express 5 — um único processo expõe todos os endpoints REST (`/api/...`). Não há decomposição em microserviços independentes; a modularização é feita por routers, middlewares, services e models.

Dados políglotes:

PostgreSQL (via Prisma): identidade, multitenancy (Tenant, User), credenciais e perfis.

MongoDB (via Mongoose): domínio operacional (ativos, manutenções, movimentações, aprovações, auditoria em documentos).

Mensageria assíncrona: publicação de eventos de domínio através de RabbitMQ (configurável por `EVENT_BROKER_DRIVER`), com circuit breaker e worker opcional para consumo.

Deploy típico: frontend na Vercel, backend e worker no Render (ver em `render.yaml`).

Modelo escolhido: arquitetura monolítica em camadas com separação lógica entre apresentação HTTP, regras de negócio (services), persistência (Prisma/Mongoose) e integrações (brokers, API de integração externa). Como visto na figura 10 na página 25. Esta escolha simplifica o deploy acadêmico e mantém coerência transacional no mesmo processo Node.

5.1.1 Segmentação da Arquitetura

A divisão lógica inspira-se em camadas semelhantes às de DDD / Clean Architecture de forma pragmática (não há camada de domínio pura isolada de frameworks em todos os módulos, mas há separação clara de responsabilidades).

Camada lógica	Localização no código	Função
Apresentação (HTTP)	<code>backend/src/routes/*.js</code>	Define verbos HTTP, monta respostas JSON, delega validação e serviços.
Aplicação / orquestração	Rotas + <code>asyncHandler</code> + middlewares	Encadeia autenticação (<code>authMiddleware</code>), autorização (<code>authorize</code>), limites de taxa e chamadas aos services.
Domínio / negócio	<code>backend/src/services/*.js</code>	Regras de negócio (ex.: aprovações, manutenções, relatórios), uso de <code>AppError</code> para falhas de domínio.
Infraestrutura	<code>backend/src/lib/prisma.js</code> , <code>mongoose.js</code> , <code>backend/src/models/*</code> , <code>backend/src/adapters/*</code>	Conexões, persistência, adaptadores para broker (RabbitMQ).
Cliente (UI)	<code>src/</code>	Views Vue, router, stores Pinia, serviço Axios.

Fonte: Produzido pelo autor.

5.2 TECNOLOGIAS UTILIZADAS

5.2.1 Frontend

As tecnologias que compõem a interface do utilizador são as seguintes:

- Vue (^3.5.32): Framework UI principal utilizando Composition API e a sintaxe <script setup>.
- Vue Router (^5.0.4): Gestão de rotas da SPA, incluindo guards de autenticação e verificação de perfis (meta.roles).
- Pinia (^3.0.4): Gestão de estado global (módulos de auth, inventory) e intermediação de chamadas à API.
- Axios (^1.14.0): Cliente HTTP configurado com withCredentials: true para suporte a cookies de sessão.
- TypeScript (~6.0.2 devDependency): Utilizado para tipagem em src/**/*.ts e suporte durante o processo de build.
- Vite (^8.0.4): Ferramenta de build e servidor de desenvolvimento para o frontend.
- @vitejs/plugin-vue (^6.0.5): Plugin de integração essencial entre o Vue e o ecossistema Vite.
- lucide-vue-next (^0.468.0): Biblioteca de ícones utilizada na interface.
- CSS: Estilização implementada em src/styles.css e blocos <style scoped> dentro dos componentes (sem dependência de Tailwind).

5.2.2 Backend

O ecossistema do servidor e lógica de negócio utiliza as seguintes ferramentas:

- Node.js: Runtime de execução (recomendado versão 18 LTS ou superior) para o servidor e execução de testes.
- Express (^5.2.1): Framework para aplicação HTTP, organização de routers e middlewares.
- dotenv (^17.4.1): Utilitário para o carregamento seguro de variáveis de ambiente.
- jsonwebtoken (^9.0.3): Implementação de JWT para gestão de sessões.
- bcryptjs (^3.0.3): Algoritmo de hash para armazenamento seguro de palavras-passe no Prisma.
- cookie-parser (^1.4.7): Middleware para leitura do cookie token enviado pelo cliente.
- cors (^2.8.6): Gestão de origens permitidas via CORS_ORIGIN.
- helmet (^8.1.0): Configuração de cabeçalhos de segurança HTTP.
- express-rate-limit (^8.3.2): Controlo de limite de pedidos para proteção de endpoints de login e integrações.
- zod (^4.3.6): Biblioteca para validação rigorosa de esquemas de entrada de dados.
- multer (^2.1.1): Middleware para gestão de upload de ficheiros.
- google-auth-library (^10.6.2): Biblioteca para validação de credenciais Google no fluxo de login.
- amqpplib (^1.0.7) : Adaptadores para comunicação com os brokers de eventos.

5.2.3 Banco de Dados

O sistema adota uma estratégia de persistência poliglota:

- PostgreSQL: Atua como base de dados relacional para gestão de tenants, utilizadores, roles e credenciais. A interação é feita via Prisma (@prisma/client e CLI 5.22.0), com definição no ficheiro backend/prisma/schema.prisma.
- MongoDB: Utilizado para o domínio operacional, incluindo ativos, manutenções, movimentações, aprovações e logs de auditoria. A interação é feita via Mongoose (^9.4.1), com modelos definidos em backend/src/models/.
- As conexões são geridas pelas variáveis de ambiente DATABASE_URL e MONGODB_URL.

5.2.4 Ferramentas de Apoio

O desenvolvimento e deploy contam com o suporte das seguintes ferramentas:

- Git: Sistema de controle de versão para o código-fonte.
- npm: Gestor de dependências e executor de scripts (npm run dev, npm test).
- concurrently (^9.2.1): Permite o arranque simultâneo do frontend e backend a partir da raiz do projeto.
- Docker / Docker Compose: Utilizado para levantar serviços auxiliares localmente (PostgreSQL, RabbitMQ) via backend/docker-compose.yml.
- Render / Vercel: Plataformas de hospedagem para backend/worker e frontend, respetivamente, utilizando ficheiros de configuração render.yaml e vercel.json.
- Postman / Insomnia / Thunder Client: Utilizados para testes manuais dos endpoints REST (/api/health, auth, CRUD).
- Cursor / VS Code: Ambientes de desenvolvimento (IDE) com assistência para escrita de código e documentação.

5.2.5 Padrões Adotados

O projeto implementa diversos padrões de desenho para garantir escalabilidade e organização:

- Middleware em cadeia: Padronização do Express para processar pedidos sequencialmente (helmet, cors, json, cookieParser) antes de atingir os routers.
- Adapter: Implementado em backend/src/adapters/eventBus/ para isolar os detalhes do RabbitMQ e Kafka; o eventBus.js atua como orquestrador que seleciona o driver correto.
- Circuit Breaker: Localizado em backend/src/utils/circuitBreaker.js, protege a aplicação contra falhas em cascata ao comunicar com os brokers.
- Singleton (Prisma Client): Implementado em backend/src/lib/prisma.js para garantir uma única instância de ligação reutilizada por todo o processo.
- Repository implícito: Uso do Prisma e Mongoose como abstração de acesso a dados; os serviços consomem os modelos sem injetar SQL diretamente nas rotas.
- Factory / HOF: Exemplo na função authorize(['ADM','GESTOR']) em middlewares/auth.js, que gera middlewares configurados conforme a necessidade.

- DTO via validação: Mapeamento de documentos para objetos JSON estáveis através de funções toDto nos services (ex.: maintenanceService.js) e validação de entrada com Zod.

5.2.6 Boas Práticas e Convenções

SOLID (exemplos no projeto):

- SRP (Single Responsibility): ficheiros de rotas por recurso (assets.js, maintenances.js); middlewares dedicados a autenticação/autorização; serviços com nomes alinhados ao caso de uso (approvalService, reportService).
- OCP (Open/Closed): novos endpoints podem ser acrescentados com novos routers em routes/ e registo em server.js sem alterar o núcleo de outros módulos.
- DIP (Dependency Inversion) — exemplo com adaptadores: o domínio publica eventos através de publishDomainEvent / publishDomainEventSafely sem importar diretamente amqp-lib ou kafka.js nos services de negócio principal; a infraestrutura concreta está nos adapters do event bus, reduzindo acoplamento.

Clean Code:

- Nomes descritivos (listMaintenancesForTenant, integrationApiKeyAuth, publishDomainEventSafely).
- Funções com propósito único nos utils (asyncHandler, AppError).
- Evitar comentários óbvios; preferir código legível e mensagens de erro úteis ao utilizador (message em JSON).

DTOs (Data Transfer Objects):

- Os services expõem objetos planos (campos necessários para a API) via funções toDto ou agregações em reportService.js, em vez de devolver documentos Mongoose crus com metadados internos.
- Zod valida o corpo de entrada (safeParse) antes de persistir — atua como contrato/DTO de entrada nas rotas que o utilizam.

Tratamento de erros e exceções:

- AppError: erros operacionais com statusCode e mensagem.
- asyncHandler: captura promessas rejeitadas nas rotas async.
- Handler global em server.js: distingue AppError, erros de upload/Prisma e devolve JSON genérico em produção (Erro interno do servidor.) sem expor stack trace ao cliente.

Versionamento semântico e Git:

- package.json raiz: version 0.0.0 (protótipo); backend: 1.0.0.
- Recomendação: usar tags v1.0.0, v1.1.0 em marcos de entrega; commits com mensagens claras (ex.: convenção Conventional Commits: feat:, fix:, docs:).

Padrão de resposta da API:

- Não existe um envelope único `{ data, error, message }` em todos os endpoints.
- Prática atual: em sucesso, muitas rotas devolvem o recurso ou lista diretamente em JSON; em erro, `{ message: '...' }` (e por vezes issues com detalhes Zod).
- Rotas de integração e relatórios podem incluir metadados (version, protocol, data) — ver `integrations.js` e respostas de `/api/reports/export/:type`.

Injeção de dependência:

- Manual: import de módulos (prisma, modelos Mongoose). Não há container IoC (tipo NestJS).

Mapeamento de objetos:

- Conversão manual nos services (toDto, agregações em `reportService.js`).

Segurança básica:

- Segredos e URLs em variáveis de ambiente (`backend/.env`, nunca commitar produção — ver `.gitignore`).
- `bcrypt` para hash de palavras-passe.
- JWT em cookie `httpOnly`; CORS restrito; rate limit em login e integrações; validação de uploads por tenant em `uploads.js`.
- `INTEGRATION_API_KEYS` ou par `INTEGRATION_API_KEY + INTEGRATION_TENANT_ID` para API de parceiros.

5.2.7 Requisitos de Infraestrutura

Ambiente de desenvolvimento

- SO: Windows 10/11, Linux ou macOS.
- Node.js: 18.x ou superior (recomendado LTS).
- npm: compatível com o lockfile do projeto.
- Memória RAM: mínimo 4 GB (8 GB recomendado com IDE + browser + Mongo + Postgres locais).
- Disco: espaço para `node_modules` e bases locais.
- Serviços: instâncias acessíveis de PostgreSQL, MongoDB e, opcionalmente, RabbitMQ (ou URL CloudAMQP em `.env`).

Produção:

- Seguir limites do plano Render (free tier) e Vercel; configurar todas as variáveis listadas em `backend/.env.example` e `docs/deploy-vercel-render.md`.

5.2.8 APIs e Integrações

O Assetra comunica com diversos serviços e plataformas para garantir a sua funcionalidade e disponibilidade:

- Google Identity / OAuth: Utilizado para o fluxo de autenticação e pré-preenchimento de dados no painel administrativo. No backend, é utilizada a biblioteca google-auth-library para validação; no frontend, a integração é feita através da variável VITE_GOOGLE_CLIENT_ID e o script GIS (Google Identity Services) quando aplicável.
- RabbitMQ (ex.: CloudAMQP): Atua como o broker AMQP principal para o disparo e gestão de eventos de domínio de forma assíncrona. A ligação é configurada via variável RABBITMQ_URL e o processamento é feito pelo worker dedicado (npm run worker:events).
- Render: Plataforma responsável pela hospedagem da API em Node.js e do worker de eventos. Inclui monitorização de disponibilidade através do endpoint de health check em /api/health.
- Vercel: Plataforma utilizada para a hospedagem e distribuição do build estático da aplicação Vue (frontend).

Adicionalmente, o sistema expõe uma API de integração própria (/api/integrations), protegida por autenticação via X-API-Key. Esta interface permite que sistemas parceiros realizem operações externas, tais como a listagem de manutenções, reatribuição de tarefas e consultas de auditoria por entidade.

5.2.9 Caracterização da API

Estilo: REST sobre HTTP/1.1 (e HTTPS em produção).

Formato de troca: JSON (Content-Type: application/json), exceto uploads (multipart/form-data) e ficheiros servidos por GET /api/uploads/:filename.

Autenticação principal: cookie httpOnly com JWT; o cliente Axios envia withCredentials: true. Pode coexistir header Authorization: Bearer em cenários legados, mas a sessão privilegia cookie.

Endpoints de exemplo: /api/auth/*, /api/assets, /api/maintenances, /api/movements, /api/approvals, /api/tasks, /api/uploads, /api/users, /api/reports/summary, /api/reports/export/:type, /api/integrations/v1/*, /api/health, /api/metrics.

5.3 REPOSITÓRIO E CÓDIGO-FONTE

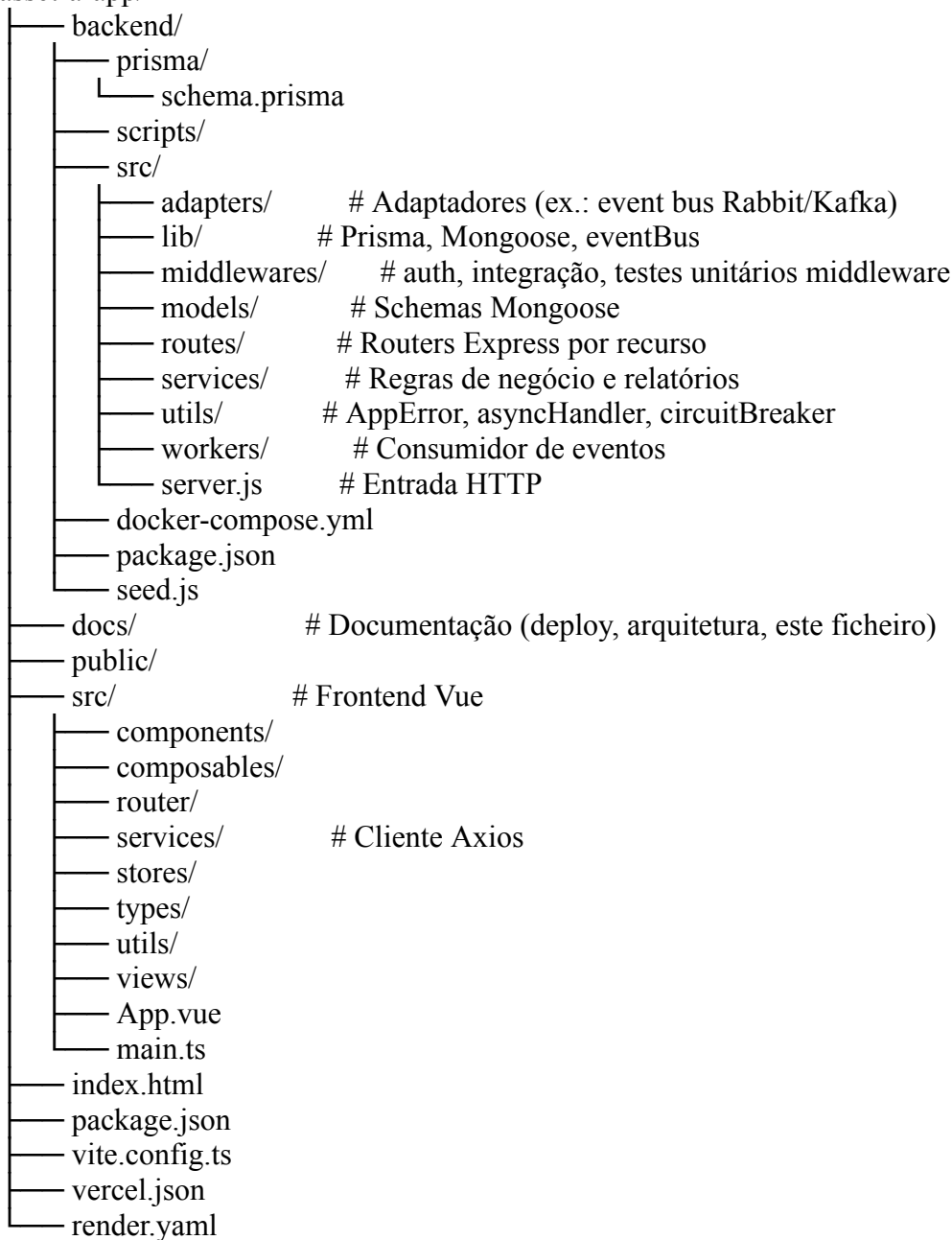
5.3.1 Localização

O código-fonte deste projeto reside no repositório Assetra no Github: (github.com).

5.3.2 Estrutura de pastas (árvore simplificada)

Plaintext

assetra-app/



5.3.3 Conteúdo das pastas principais

Pasta	Conteúdo
src/	Interface Vue: páginas, componentes, estado Pinia, router, chamadas HTTP.
backend/src/routes/	Definição REST e encadeamento de middlewares.

backend/src/services/	Lógica de negócio, DTOs, integração com Prisma/Mongoose, relatórios, auditoria.
backend/src/models/	Esquemas e modelos Mongoose.
backend/src/adapters/	Implementações concretas desacopladas (brokers).
backend/prisma/	Schema e migrações SQL.
docs/	Guias de deploy, padrões de arquitetura, documentação técnica.
public/	Assets estáticos servidos pelo Vite.

Testes: ficheiros *.test.js junto aos módulos (ex.: authService.test.js, integrationAuth.test.js); comando npm test na pasta backend/.

Manual do Usuário

6 MANUAL DO USUÁRIO

Este guia é para quem vai usar o sistema no dia a dia, sem conhecimento técnico de programação. O objetivo é que consiga concluir tarefas com segurança, passo a passo.

Antes de começar (o que precisa)

Para garantir que o Assetra funcione corretamente, verifique os seguintes itens:

- Computador ou tablet: Um computador com rato e teclado é mais confortável para preencher formulários longos.
- Navegador: Utilize o Google Chrome ou Microsoft Edge (versões recentes de 2026). Evite navegadores muito antigos para evitar erros de visualização.
- Ligação à Internet: Necessária pois o sistema está na nuvem. Em teste na sua máquina, o técnico de informática indica o endereço a abrir.
- Resolução de ecrã: O ideal é 1366×768 ou superior (como 1920×1080). Se o ecrã for pequeno, use o zoom em 100% no navegador.
- Dados de acesso: E-mail e palavra-passe fornecidos pela sua organização.

Para fins de teste, use a organização default no campo da organização (slug).

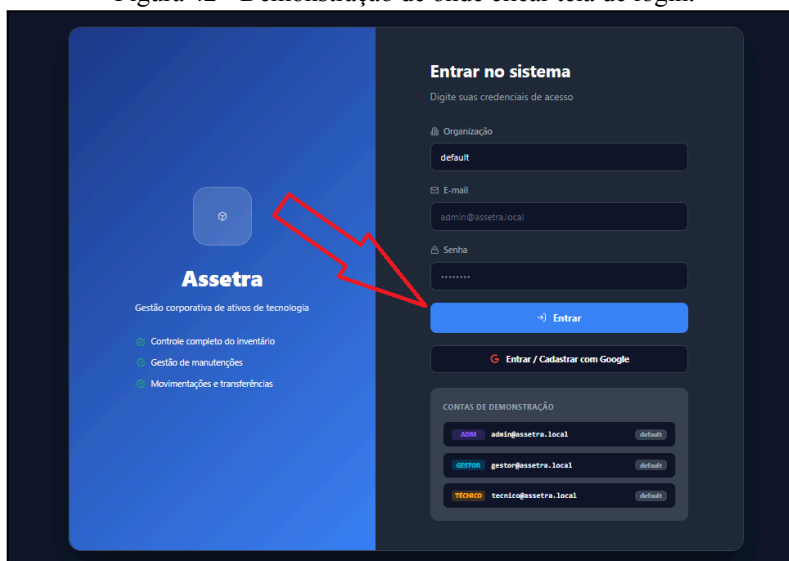
- Perfil Administrador: * E-mail: admin@assetra.local | Palavra-passe: senha123
- Perfil Gestor: * E-mail: gestor@assetra.local | Palavra-passe: senha123
- Perfil Técnico: * E-mail: tecnico@assetra.local | Palavra-passe: senha123

Organização “Acme” (demonstração de duas empresas): se no início de sessão escolher a organização acme, o administrador usa o mesmo e-mail admin@assetra.local com a palavra-passe AcmeDemo@12345. Em uso normal, cada empresa tem os seus próprios utilizadores.

Como entrar no sistema

1. Abra o navegador e escreva o endereço do Assetra que lhe foi indicado (na empresa).
2. Na página de início de sessão, preencha o e-mail e a palavra-passe ou entre com google no botão entrar com google abaixo do botão entrar.
3. Se aparecer o campo da organização, escreva o nome curto que lhe deram (por exemplo default) — confirme com o administrador.
4. Clique no botão Entrar. Onde clicar:

Figura 42 - Demonstração de onde clicar tela de login.



Fonte: Produzido pelo autor.

Dica: Se a mensagem disser que os dados estão incorretos, volte a escrever com atenção. Lembre-se que maiúsculas fazem diferença na palavra-passe!

Estrutura do manual por tarefas

Este manual está organizado por objetivos: “O que quero fazer?” — siga os números na ordem.

Tarefa A — Ver o painel inicial depois de entrar

1. Após Entrar, verá o painel com resumo da sua função (administrador, gestor ou técnico).
2. No lado esquerdo está o menu. Use-o para saltar entre áreas (Ativos, Manutenções, etc.).

Tarefa B — Como registar um novo ativo (equipamento)

Quem costuma fazer isto: Normalmente gestor ou administrador.

1. No menu, abra Ativos de TI.
2. Clique em Novo Ativo/ abra o formulário de novo ativo.
3. Preencha pelo menos: identificação (etiqueta), descrição, setor e estado.
4. Se quiser atribuir o equipamento a uma pessoa, escolha o e-mail sugerido na lista.
5. Clique em Cadastrar (o botão final do formulário).

Nota: Se um campo obrigatório faltar, o sistema avisa e não grava até corrigir.

Tarefa C — Como registar uma movimentação (mudança de local)

Quem costuma fazer isto: gestor ou administrador.

1. Abra Movimentações no menu.
2. Indique o ativo (etiqueta), origem, destino e responsável.
3. Use as sugestões que aparecem ao escrever, se existirem, para evitar erros de digitação.
4. Clique em Registrar / Guardar.

Tarefa D — Como abrir um pedido de manutenção

Quem pode: Depende da configuração da equipa; geralmente gestor ou técnico.

1. Abra Minhas Solicitações.
2. Clique em Solicitar Manutenção.
3. Escolha o ativo, o tipo de manutenção, descreva o problema e a prioridade.
4. Se for preciso atribuir um técnico, indique o e-mail do técnico nas sugestões.
5. Pode anexar fotos (imagens ou PDF) se o formulário o permitir.
6. Confirme com Enviar para aprovação.

Tarefa E — Como aprovar ou reprovar um pedido

Perfil: Gestor ou Administrador.

1. Abra Aprovações no menu.
2. Veja a lista de pedidos pendentes.
3. Abra o pedido que quer tratar.
4. Escolha Aprovar ou Reprovar (botões ou ações claras no ecrã).
5. Se lhe for pedida a palavra-passe para confirmar, escreva a sua palavra-passe de utilizador e confirme.

Atenção: Em fluxos de manutenção, uma reprovação pode devolver o trabalho ao técnico ou reencaminhar para outro técnico.

Tarefa F — Como o técnico vê tarefas e envia conclusão

1. Entre com o perfil técnico.
2. Abra Execução técnica (ou nome semelhante no menu).
3. Veja a lista de ordens atribuídas a si.
4. Avance o estado conforme os botões disponíveis (por exemplo avançar etapa).
5. Quando pedir relatório para validação, preencha a descrição resumida e confirme com a sua palavra-passe se o sistema pedir.

Tarefa G — Como usar o assistente de solicitações (com fotos)

1. Abra Minhas solicitações no menu.
2. Siga os passos indicados no ecrã (assistente).
3. Anexe ficheiros quando pedido (fotos do equipamento, documentos).
4. Conclua com Enviar ou Submeter.

Tarefa H — Como ver relatórios e exportar dados

Perfil: Gestor ou Administrador.

1. Abra Relatórios no menu.
2. Opcional: abra Filtros, escolha datas e setor, e clique em Aplicar.
3. Para descarregar um relatório, clique no cartão desejado (por exemplo ativos ou manutenções) e guarde o ficheiro que o navegador oferece.

Glossário de mensagens (o que o sistema quer dizer)

- “Credenciais inválidas” / “Dados de login inválidos”: O e-mail ou a palavra-passe não coincidem. Verifique o teclado (CAPS LOCK), espaços extras e se a organização está correta.
- “Acesso negado” / “Permissão insuficiente”: A sua função não tem autorização para esta ação. Contacte um gestor ou administrador.
- “Sessão inválida”: O sistema perdeu a conexão com a sua sessão. Basta Sair e Entrar novamente.
- “Dados inválidos”: Algum campo obrigatório está vazio ou no formato errado. Corrija os campos assinalados.
- “Já existe uma validação pendente...”: Para esta ordem de manutenção, já foi enviado um pedido. Aguarde a decisão do gestor.
- “Senha incorreta” (na confirmação): A palavra-passe escrita para confirmar uma ação sensível está errada. Tente novamente.
- “Não foi possível carregar...” / página em branco: Pode ser falha de internet ou o servidor está offline. Atualize a página (F5).
- “Muitas tentativas de login”: Bloqueio temporário por segurança. Aguarde alguns minutos antes de tentar de novo.

Perguntas frequentes (FAQ)

1. Esqueci a minha palavra-passe. O que faço? Peça ao administrador da sua organização para repor ou criar uma nova. O sistema não dispõe de envio automático de recuperação por e-mail neste momento.

2. O botão Guardar / Entrar não faz nada. Verifique se todos os campos obrigatórios (marcados com * ou cor diferente) estão preenchidos. Procure por mensagens de erro em vermelho.
3. Não vejo o menu que o colega descreveu. As opções mudam conforme a função (administrador, gestor, técnico). Peça ao administrador para confirmar o seu perfil.
4. As sugestões de e-mail ou nomes não aparecem. Comece a escrever uma letra para filtrar os resultados. Se continuar vazio, pode ser falta de permissão para listar utilizadores.
5. O sistema abre mas diz que não está a responder. Confirme a sua ligação à Internet e tente atualizar a página. Se o erro persistir, contacte o suporte informático com o endereço (URL) que está a usar.

Referências Bibliográficas

7. REFERÊNCIAS

Esta seção é obrigatória e lista todos os materiais consultados. Devem ser organizadas em ordem alfabética pelo sobrenome do autor e com espaçamento simples (com uma linha em branco entre elas).

- Para Livros:
SOBRENOME, Nome. Título do livro: subtítulo. Edição. Local: Editora, Ano.

Exemplo:

MARTIN, Robert C. Código limpo: habilidades práticas do software Agile. Rio de Janeiro: Alta Books, 2011.

- Para documentações online:
AUTOR/ORGANIZAÇÃO. Título da página/documentação. Ano. Disponível em: <URL>. Acesso em: dia mês ano.

Exemplo:

PYTHON SOFTWARE FOUNDATION. Python 3.12.2 documentation. 2024. Disponível em: <https://docs.python.org/3/>. Acesso em: 25 fev. 2026.

- para sites de Artigos Técnicos:
SOBRENOME, Nome. Título do artigo. Nome do Site, Ano. Disponível em: <URL>. Acesso em: dia mês ano.

Exemplo:

FOWLER, Martin. Presentation Domain Separation. martinowler.com, 2015. Disponível em: <https://martinfowler.com/bliki/PresentationDomainSeparation.html>. Acesso em: 25 fev. 2026.

- Para ferramentas:
AUTOR (Empresa ou Desenvolvedor). Nome do software: subtítulo (se houver). Versão (se aplicável). Local (cidade, se souber): Fabricante/Editora, Ano. Disponível em: <URL>. Acesso em: dia mês ano.

Exemplo:

MICROSOFT. Visual Studio Code. Versão 1.85. Redmond: Microsoft Corporation, 2023. Disponível em: <https://code.visualstudio.com>. Acesso em: 2 mar. 2026.

Apendice

8. APÊNDICE

Inclua nesta seção todos os materiais complementares produzidos por si. É obrigatória a inclusão de todo o código fonte do projeto na linguagem desenvolvida, scripts de criação, manipulação de base de dados, logs para comprovação dos testes, scripts de marcação de texto referente a ferramentas diversas como LaTeX, PlantUML, etc.

Utilize fonte monospaçada tamanho 10 para blocos de código.

O Apêndice é o local onde o aluno coloca materiais que ele mesmo produziu, mas que são demasiado extensos ou detalhados para figurar no meio dos capítulos principais, pois poderiam interromper a fluidez da leitura.

Diferente do Anexo (que é material de terceiros), o Apêndice é autoria do próprio aluno.